Efficient Resilience and Responsibility for Graph Queries

Dazhuo Qiu
Lyon 1 University
CNRS LIRIS
Lyon, France
dazhuo.qiu@univ-lyon1.fr

Angela Bonifati
Lyon 1 University
CNRS LIRIS & IUF
Lyon, France
angela.bonifati@univ-lyon1.fr

Andrea Mauri
Lyon 1 University
CNRS LIRIS
Lyon, France
andrea.mauri@univ-lyon1.fr

Fateh Barca
Lyon 1 University
CNRS LIRIS
Lyon, France
ahmed-fateh.barca@etu.univ-lyon1.fr

***Abstract*—Resilience and responsibility explain a query answer by identifying which parts of the data support it and how robust it is under interventions on the data. For graph queries, existing work considers interventions that delete single edges. We introduce pattern-based resilience and responsibility, which generalize these interventions to *intervention units*: matches of arbitrarily complex, user-defined graph patterns, such as molecular rings or laundering typologies, and show how to compute them efficiently in both memory and time. This setting raises new challenges: units overlap query matches and each other, so materializing all unit–match pairs, as set cover and greedy do, can exhaust memory; an effective unit may not be contained in any query match; and candidate units are combinatorially many. We develop MILP formulations that link units to the query matches they destroy only through shared edges, so their size grows linearly with the numbers of units and query matches. We further derive LP-relaxation-based randomized rounding algorithms with logarithmic approximation guarantees. Experiments on real and synthetic property graphs, including a billion-edge graph, show solution sizes close to exact optima and optimized greedy. Where units overlap many query matches, LP rounding uses up to $108\times$ less peak memory than greedy and is up to $19\times$ faster, with somewhat larger solutions, and it completes a workload on which greedy runs out of memory.**

I. Introduction

Property graphs model interconnected entities in finance, security, social media [1], and life sciences [2]. Vertices and edges carry labels and key–value properties, representing both topology and attributes [3]. Systems include Neo4j [4], Amazon Neptune [5], Oracle PGX [6], SAP HANA Graph [7], RedisGraph [8], Sparksee [9], and Kuzu [10]; GQL and SQL/PGQ standardize graph querying [3], [11].

Graph query languages such as openCypher, GQL, and SQL/PGQ provide a declarative interface for querying property graphs through *graph pattern matching*. A graph query specifies a pattern over vertices and edges, together with label constraints and property predicates. Evaluating the query over a property graph returns the matches of the pattern, where each *match* is a subgraph instance of the query pattern.

Given the matches returned by a graph query, users are often interested not only in the answer itself, but also in which graph structures support it. Existing work studies the *resilience* of regular path queries, i.e., the minimum number of edge deletions after which the query has no match [12]. Beyond single edges, explanations and actions in many domains are expressed through recurring structures: medicinal chemistry organizes molecules into rings, linkers, and frameworks [13], [14], and anti-money-laundering analysis groups connected transactions into typologies [15], [16]; similar structures arise as laundering motifs [17], functional groups [18], dependency and attack patterns [19]–[22], and user–item paths [23], [24].

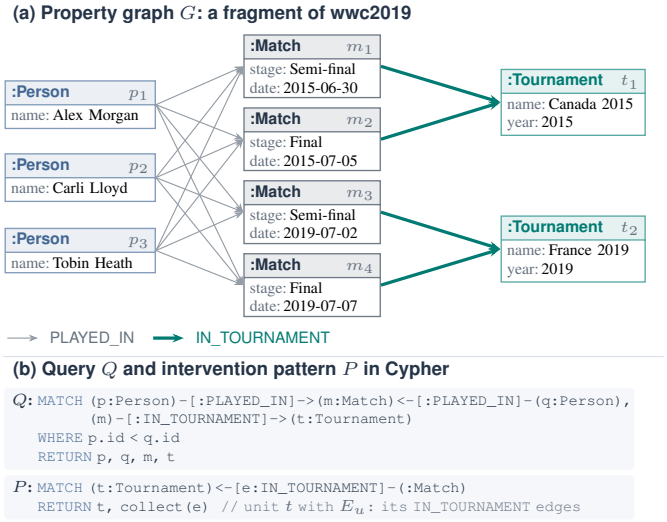


Fig. 1. Running example. (a) A fragment of wwc2019 [25]: players p_1–p_3 each played the semi-final and the final of the 2015 and 2019 World Cups. (b) Q retrieves two players of the same match with the tournament of the match; each intervention unit of P is a tournament with its IN_TOURNAMENT edges, so it overlaps every query match of its games. Since units can overlap many query matches, our formulations link units and matches only through shared edges instead of materializing unit–match pairs, which lets LP rounding use up to $108\times$ less peak memory than greedy while being up to $19\times$ faster (Section VI-C).

Motivated by this, we introduce *pattern-based resilience* (P-RES) and *pattern-based responsibility* (P-RSP). Given a property graph G, a graph query Q, and an intervention pattern P, an *intervention unit* is a match of P in G. Selecting a unit deletes all of its edges, and an edge shared by several selected units is deleted once; pattern-based interventions are thus *bulk interventions*. P-RES returns a minimum set of intervention units whose deletion eliminates all matches of Q, called a *minimum intervention set*. Given a unit u, P-RSP

returns a minimum *contingency set* S such that some match of Q survives the deletion of S, but none survives when u is additionally deleted; the responsibility of u is $1/(1+|S|)$, or zero if no contingency set exists. P-RES measures the vulnerability of the query answer, whereas P-RSP measures the importance of a given unit, including units that belong to no minimum intervention set. Their tuple-based counterparts are likewise studied together [26]–[28].

Example 1. Figure 1(a) shows a fragment of the wwc2019 property graph: three players p_1–p_3 who each played the semi-final and the final of two World Cups, t_1 (2015) and t_2 (2019). The query Q (Figure 1(b)) retrieves two players who played in the same match, together with the tournament of the match; it has 12 matches on the fragment, three for each of $m_1, \ldots, m_4$. The intervention pattern P is a single vertex: a tournament together with its IN_TOURNAMENT edges, giving two intervention units t_1 and t_2. $\{t_1, t_2\}$ is the minimum intervention set, so the pattern-based resilience of Q is two: instead of the four IN_TOURNAMENT edges it deletes, it names the two tournaments that support the query. For P-RSP, t_1 has responsibility $1/2$, with the contingency set $\{t_2\}$.

• **Challenges**. Pattern-based interventions raise three challenges. *(C1) Memory and time.* A unit can overlap many query matches, so the unit–match pairs materialized by set cover or greedy can reach the product of the numbers of units and query matches; on one of our workloads, greedy faces 9.3×10^9 such pairs and runs out of memory on a 512 GB machine. *(C2) Non-local optima.* Units overlap query matches and each other, and selected units delete their shared edges only once; an optimal unit need not even be contained in any query match, so optimal solutions cannot be found by enumerating query matches.

Example 2. Recall Example 1. The intervention unit t_1 is not contained in any query match: each query match contains only one of its two IN_TOURNAMENT edges. Yet t_1 belongs to the minimum intervention set, since it partially overlaps with six query matches. This shows that optimal interventions must be sought globally over all matches of the intervention pattern instead of query matches.

(C3) Combinatorial search space. The candidate units are all matches of P in G, whose number grows polynomially in the edges of G with the edges of P as exponent; responsibility must also choose which query match survives.

• **Our Contributions**. Our main contributions are as follows.

1) We introduce P-RES and P-RSP, whose intervention units are matches of arbitrarily complex graph patterns, and show that both are NP-complete even when the query and the intervention pattern are fixed and have at most two edges (Section III).
2) (C1, C2) We present exact MILP formulations whose size is linear in the numbers of units and query matches: they store the edges of each unit and of each query match once, whereas a direct set-cover formulation stores every unit–match pair sharing an edge. Edge-deletion variables link unit selection to query-match destruction, capturing overlapping units and shared edges (Section IV).
3) (C3) We develop LP-relaxation-based randomized rounding with deterministic feasibility repair. For P-RES, it gives an expected $O(\log n)$ approximation for fixed queries, where n is the number of query matches, with an analysis over edges rather than unit–match pairs. For P-RSP, we decompose the problem by the surviving query match, obtain the same guarantee, and exactly prune equivalent and dominated candidates (Section V).
4) (C1) We evaluate our methods on six real property graphs and a synthetic graph with 1.78 billion edges (Section VI). LP rounding returns sets close to exact optima and to optimized greedy. Where units overlap many query matches, greedy materializes up to $411\times$ more unit–match pairs than our formulations store unit and match edges, and LP rounding needs up to $108\times$ less peak memory and is up to $19\times$ faster (e.g., 5.0 versus 231 GiB and 51 versus 954 seconds), with up to 27% larger sets; it also finishes the workload on which greedy runs out of memory.

II. Preliminaries

Property Graphs. A *property graph* is a structure $G = (V, E, \eta, \lambda, \nu)$ where: (i) V is a finite set of vertices (nodes) and E is a finite set of edges with $V \cap E = \emptyset$; (ii) $\eta : E \to V \times V$ maps each edge to an ordered pair of vertices; (iii) $\lambda : (V \cup E) \to \mathcal{P}(\mathcal{L})$ assigns a finite set of labels to each vertex or edge; and (iv) $\nu : (V \cup E) \times \mathcal{K} \to \mathcal{N}$ is a partial function assigning property values, i.e., key–value pairs, to vertices and edges [29], [30].

Example 3. In Figure 1(a), the property graph represents players, matches, and tournaments as vertices with labels Person, Match, and Tournament, and represents that a player played in a match and that a match belongs to a tournament as edges with labels PLAYED_IN and IN_TOURNAMENT. Each vertex is drawn with its label and some of its properties, such as the name of a player and the stage and date of a match.

Graph Query and Graph Pattern. We consider graph queries over property graphs that retrieve subgraphs satisfying a given structural specification and a set of predicates over vertex and edge properties. The structural and predicate constraints of such a query are represented by a graph pattern $Q : \Pi = (V_\Pi, E_\Pi, \phi_V, \phi_E)$, where V_Π and E_Π denote pattern vertices and edges, and ϕ_V and ϕ_E are predicates over vertex and edge labels or properties. A match of Q in a property graph G is a subgraph of G that satisfies both the structure induced by (V_Π, E_Π) and the predicates specified by ϕ_V and ϕ_E. Evaluating a graph query amounts to finding all such matches of its associated graph pattern.

Example 4. Figure 1(b) writes the query in Cypher. Its graph pattern has vertices p, q with label Person, m with label Match, and t with label Tournament, with PLAYED_IN edges from p and q to m and an IN_TOURNAMENT edge

from m to t; the predicate on the property id counts each pair of players once. Section IX-I gives it formally as Q_W.

Matches and Query Answer. A *match* of a graph pattern query Q in a property graph G is an embedding $h = (h_V, h_E)$ that maps query vertices and query edges to vertices and edges in G, while preserving adjacency, labels, and property predicates. That is, for every query edge $(x, y) \in E_\Pi$, the mapped edge $(h_V(x), h_V(y))$ must exist in G, and all predicates in ϕ_V and ϕ_E must be satisfied. We assume injective matching, i.e., distinct query vertices are mapped to distinct graph vertices. Let $\mathcal{M}_Q(G)$ denote the set of all matches of Q in G. If Q returns full embeddings, then the query answer is $Ans(Q, G) = \mathcal{M}_Q(G)$. If Q projects selected query variables, then $Ans(Q, G)$ is the projection of all matches onto those variables. For a Boolean query, the answer is true iff $\mathcal{M}_Q(G) \neq \emptyset$.

Example 5. In Figure 1(a), mapping (p, q, m, t) to (p_1, p_2, m_1, t_1) gives a match w_1 if $\nu(p_1, \mathsf{id}) < \nu(p_2, \mathsf{id})$, and to (p_2, p_1, m_1, t_1) otherwise. The query thus has 12 matches, one for each pair of players and each of $m_1, \ldots, m_4$.

III. Pattern-based Resilience and Responsibility

In causality theory, an *intervention* denotes an action that modifies part of a system, and its effect is assessed by evaluating the post-intervention outcome [31], [32]. In relational databases, interventions are typically modeled as tuple deletions [26]–[28], [33]. In property graphs, however, interventions naturally arise over graph structures, such as edges [12], [34] and subgraphs. To the best of our knowledge, we are the first to model interventions as pattern-induced subgraph deletions. Each intervention unit is a matched subgraph, represented by its set of edges, whose deletion may change the query outcome. This formulation covers single-edge to arbitrarily complex subgraph interventions.

Definition 1 (Intervention Pattern and Intervention Unit)**.** An *intervention pattern* P is a graph pattern, which can retrieve subgraph matches from graph database G. The retrieved subgraph matches are referred to as *intervention units*. An *intervention unit* u is a subgraph match of P in G. Therefore, the intervention universe is $\mathcal{M}_P(G)$. For each intervention unit $u \in \mathcal{M}_P(G)$, let $E_u \subseteq E$ denote the set of edges selected by u. When P consists of a single vertex, each intervention unit is a vertex v satisfying the predicates of P, and E_u is the set of edges incident to v, optionally restricted to given edge labels. A *candidate intervention set* is any set $S \subseteq \mathcal{M}_P(G)$. For such S, let $D(S) = \bigcup_{u \in S} E_u$ be the set of deleted edges, and let $G \setminus S$ denote G after deleting the edges in $D(S)$; vertices are retained, and an edge shared by several selected units is deleted once.

A. Resilience for Graph Queries

We now define pattern-based resilience (P-RES) for graph queries. This framework captures both atomic edge interventions and structured subgraph interventions. Atomic edge intervention is obtained when the intervention pattern selects a single edge. More generally, a matched intervention pattern may select several connected edges, such as edges belonging to a path or a sub-graph, so that an intervention unit corresponds to a structured region of the graph rather than to a single edge.

Definition 2 (Pattern-based Resilience (P-RES))**.** Let G be a property graph, Q be a graph query, and P be an intervention pattern. The *Pattern-based resilience* of Q on G with respect to P is

$$\text{P-RES}(Q, G, P) = \min_{S \subseteq \mathcal{M}_P(G)} |S| \quad \text{s.t.} \quad \mathcal{M}_Q(G \setminus S) = \emptyset.$$

A candidate intervention set S satisfying the above condition is called a *valid intervention set*. An optimal valid intervention set is called a *minimum intervention set*.

Example 6. In Figure 1, the tournament pattern P has two units t_1 and t_2 with $E_{t_1} = \{(m_1, t_1), (m_2, t_1)\}$ and $E_{t_2} = \{(m_3, t_2), (m_4, t_2)\}$. Selecting t_1 destroys the six query matches of m_1 and m_2, but those of m_3 and m_4 survive. Hence $S^* = \{t_1, t_2\}$ is the minimum intervention set, with $|S^*| = 2$ and $|D(S^*)| = 4$. Under the single-edge pattern, the minimum intervention set consists of the same four edges, with $|S^*| = |D(S^*)| = 4$.

Theorem 1 (Hardness of P-RES)**.** *The decision version of P-RES is NP-complete even when Q is a fixed two-edge query and P is a fixed single-edge intervention pattern.*

Proof sketch. Membership in NP holds because Q and P are fixed and their matches can be enumerated in polynomial time. For hardness, start with a Maximum Directed Cut instance $(D = (V, A), B)$ [35], [36]. Replace every arc (a, b) by a directed path $a \to x_{ab} \to y_{ab} \to b$ through fresh vertices, producing an oriented graph D'. For any fixed assignment of the original vertices to the two sides, a replacement path contributes at most two cut arcs when a is on the source side and b on the target side, and at most one otherwise; both bounds are attainable independently. Therefore $\mathrm{MaxDiCut}(D') = |A| + \mathrm{MaxDiCut}(D)$. Use $G = D'$, let Q be a fixed directed two-edge path under injective matching, and let P select a single edge. Put $B' = |A| + B$ and $k = |E(D')| - B'$. A directed cut of size at least B' leaves no two-edge path if all non-cut edges are deleted. Conversely, if at most k deletions leave no query match, a vertex cannot have both an incoming and an outgoing remaining arc: since D' has neither loops nor reciprocal arcs, those two arcs would give three distinct vertices and hence an injective query match. Put every remaining arc's tail on the source side and all other vertices on the target side. Every remaining arc crosses this cut, whose size is at least B'. Thus the resilience decision problem is NP-hard even for these fixed patterns. □

B. Responsibility for Intervention Units

We next define pattern-based responsibility (P-RSP), which measures how strongly a fixed intervention unit contributes to the query answer. The idea is to ask whether there exists a contingency set such that the query still holds after applying it, but becomes false once the fixed intervention unit is

additionally applied. The smaller the required contingency set, the larger the causal responsibility of the fixed unit.

Definition 3 (Pattern-based Responsibility (P-RSP))**.** Let G be a property graph, Q be a graph query, P be an intervention pattern, and let $u^\circ \in \mathcal{M}_P(G)$ be a fixed intervention unit. A set $S \subseteq \mathcal{M}_P(G) \setminus \{u^\circ\}$ is called a *contingency set* for u° if

$$\mathcal{M}_Q(G \setminus S) \neq \emptyset \quad \text{and} \quad \mathcal{M}_Q\big(G \setminus (S \cup \{u^\circ\})\big) = \emptyset.$$

The *pattern-based responsibility* of u° for Q on G with respect to P is

$$\textsf{P-RSP}(Q, G, P, u^\circ) = \begin{cases} \dfrac{1}{1 + \min |S|}, & \text{if } S \text{ exists,} \\ 0, & \text{otherwise,} \end{cases}$$

where S ranges over contingency sets for u°. We call u° an *actual cause* if there exists a contingency set. In particular, u° is a *counterfactual cause* if $S = \emptyset$ [28].

Example 7. On the same graph and pattern, let $u^\circ = t_1$. Deleting t_1 alone leaves the query matches of m_3 and m_4, so a contingency set is required. With $S = \{t_2\}$, the query matches of m_1 and m_2 survive, and additionally deleting t_1 destroys them. Hence the responsibility of t_1 is $1/2$. Under the single-edge pattern, the edge (m_1, t_1) requires the other three IN_TOURNAMENT edges as a contingency set and has responsibility $1/4$. A unit can also have positive responsibility without belonging to any minimum intervention set; Section IX-D reports such units on molecules (Table IV).

Theorem 2 (Hardness of P-RSP)**.** *The decision version of P-RSP is NP-complete with fixed intervention unit u°, even for a fixed two-edge query and a fixed single-edge intervention pattern.*

Proof sketch. For fixed patterns, a guessed contingency can be verified by enumerating matches in polynomial time. For hardness, take the fixed-pattern resilience instances from Theorem 1. Add a vertex-disjoint directed two-edge path $a \to b \to c$ and designate its first edge as u°. The connected query cannot create matches spanning the two components. Any resilience solution of size at most k in the original component leaves the fresh path intact; additionally deleting u° destroys the last match. Conversely, any contingency for u° must eliminate every original match, since u° affects none of them. It must leave the fresh path intact to witness necessity, so it cannot delete the other fresh edge. Restricting that contingency to the original component therefore gives a resilience solution of size at most k. □

IV. MILP Formulation for P-RES & P-RSP

We present Mixed Integer Linear Program (MILP) formulations for pattern-based query resilience (P-RES) and responsibility (P-RSP). To address the challenges raised in the pattern-based setting, we introduce an edge variable to track the overlapping relationship between intervention units and query matches, and between intervention units themselves. Let $U = \mathcal{M}_P(G)$ be the set of intervention units, with $m = |U|$ and let $W = \mathcal{M}_Q(G)$ be the set of query matches, with $n = |W|$. For every intervention unit $u \in U$, let $E_u \subseteq E$ be the edge set of u. For every query match $w \in W$, let $E_w \subseteq E$ be the edge set of w.

$$\min \quad \sum_{u \in U} x_u \tag{1}$$

$$\text{s.t.} \quad d_e \leq \sum_{\substack{u \in U \\ e \in E_u}} x_u \qquad \forall e \in E, \tag{2}$$

$$d_e \geq x_u \qquad \forall u \in U,\ \forall e \in E_u, \tag{3}$$

$$\sum_{e \in E_w} d_e \geq 1 \qquad \forall w \in W, \tag{4}$$

$$x_u \in \{0, 1\} \qquad \forall u \in U, \tag{5}$$

$$0 \leq d_e \leq 1 \qquad \forall e \in E. \tag{6}$$

A. MILP for P-RES

The P-RES MILP is given in (1)–(6).

Variables. The P-RES MILP requires two variables to keep track of the intervention units and ensure side effects are counted. For each intervention unit $u \in U$, introduce a binary variable **(i)** $x_u \in \{0, 1\}$, where $x_u = 1$ means that u is selected. For each edge $e \in E$, introduce a fractional variable **(ii)** $0 \leq d_e \leq 1$. For binary x, constraints (2) and (3) force $d_e = 1$ if a selected unit contains e and $d_e = 0$ otherwise, so d_e is integral without an integrality declaration. Therefore, d_e is used to track the selection of edges.

Constraints. Constraint (2) ensures that if an edge is selected, it should be included in at least one intervention unit. Constraint (3) ensures that if an intervention unit is selected, then all its edges should be selected. Constraints (2) and (3) link selected intervention units to deleted edges. Constraint (4) enforces suppression of query matches. For every query match $w \in W$, at least one edge in E_w must be deleted. Since a query match is destroyed once any of its required edges is deleted, these constraints ensure that all query matches are eliminated.

Objective. The objective of P-RES MILP is to minimize the number of intervention units to delete.

Theorem 3 (Correctness of P-RES MILP)**.** *The optimal value of the P-RES MILP is equal to the minimum number of intervention units whose deletion destroys all query matches in W.*

Intuitively, the linking constraints make $d_e = 1$ exactly for the edges deleted by the selected units, so feasible solutions and valid intervention sets correspond one to one with equal cost (proof in Section IX-B).

Example 8. Consider the example shown in Figure 2. To eliminate all seven matches $w_1, \ldots, w_7$ of the 4-cycle query Q, assume we select set $S = \{u_1, u_2, u_3\}$, then by constraint (2) and (3), edges $(4, 9)$, $(4, 5)$, $(5, 9)$, $(5, 10)$, $(9, 10)$, $(4, 10)$ are

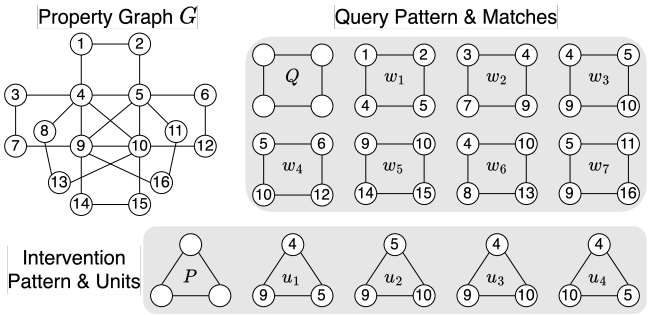


Fig. 2. A property graph G (we only show the topology of G for visualization simplicity.) For a 4-cycle graph query Q, seven query matches are returned as $w_1, \ldots, w_7$. The intervention pattern P is a 3-cycle (triangle) pattern with four intervention matches $u_1, \ldots, u_4$.

selected and all their d_e are set to 1. Constraint (4) is then satisfied since all seven matches have at least one $d_e = 1$. Since any subset of S will not satisfy constraint (4), the optimum of P-RES MILP is three, explicitly $\{u_1, u_2, u_3\}$.

Single edges and overlapping patterns. For a one-edge intervention pattern, each deletable edge e corresponds to one unit u_e with $E_{u_e} = \{e\}$. Both linking constraints then give $d_e = x_{u_e}$, and substituting yields $\min \sum_e x_{u_e}$ subject to $\sum_{e \in E_w} x_{u_e} \geq 1$ for every match, i.e., the established edge-deletion resilience ILP [26]; the same substitution recovers edge responsibility with the survivor constraints of Section IV-B. For multi-edge patterns, the feasible actions and the objective differ: selecting a unit deletes *all* its edges, selected units may overlap, and a shared edge is deleted once. Grouping an optimal edge cut into units therefore need not minimize the number of units and may delete additional edges; on MUTAG molecule 47 (Section IX-D), one carbon ring (six bonds) is optimal under the pattern cost, whereas the edge optimum deletes two bonds. The linking constraints (2)–(3) capture exactly this edge union.

B. MILP for P-RSP

Instead of asking for a minimum set to eliminate all query matches as P-RES, the P-RSP asks for a minimum set to enable a given intervention unit to be an *actual cause*, i.e., all query matches are eliminated iff the given unit is deleted.

Given an intervention unit u°, let $b_e = 1$ if $e \in E_{u^\circ}$, and $b_e = 0$ otherwise. Then b_e records whether edge e would be deleted by the fixed candidate cause u°. The responsibility MILP is given in (7)–(16).

Variables. For each intervention unit $u \in U \setminus \{u^\circ\}$, introduce a binary variable **(i)** $x_u \in \{0, 1\}$, where $x_u = 1$ means that u is selected into the contingency set. For each edge $e \in E$, introduce a fractional variable **(ii)** $0 \leq d_e \leq 1$, where d_e are forced to be $\{0, 1\}$ with the same reason as in P-RES MILP (§ IV-A). It tracks whether e is deleted by the contingency set. For each query match $w \in W$, introduce a fractional variable **(iii)** $0 \leq z_w \leq 1$, z_w is used to track the query matches that *cannot* be deleted by the contingency set, but can be deleted after deleting u°. Briefly speaking, we can choose any query match w with $z_w > 0$ as the one surviving query match. A full explanation is in Section IX-A.

$$\min \quad \sum_{u \in U \setminus \{u^\circ\}} x_u \tag{7}$$

$$\text{s.t.} \quad d_e \leq \sum_{\substack{u \in U \setminus \{u^\circ\} \\ e \in E_u}} x_u \qquad \forall e \in E, \tag{8}$$

$$d_e \geq x_u \qquad \forall u \in U \setminus \{u^\circ\},\ \forall e \in E_u, \tag{9}$$

$$\sum_{e \in E_w} (d_e + b_e) \geq 1 \qquad \forall w \in W, \tag{10}$$

$$\sum_{w \in W} z_w = 1, \tag{11}$$

$$z_w \leq \sum_{e \in E_w} b_e \qquad \forall w \in W, \tag{12}$$

$$z_w + d_e \leq 1 \qquad \forall w \in W,\ \forall e \in E_w, \tag{13}$$

$$x_u \in \{0, 1\} \qquad \forall u \in U \setminus \{u^\circ\}, \tag{14}$$

$$0 \leq d_e \leq 1 \qquad \forall e \in E, \tag{15}$$

$$0 \leq z_w \leq 1 \qquad \forall w \in W. \tag{16}$$

Constraints. Constraints (8) and (9) serve the same purposes as in P-RES MILP (§ IV-A), one difference is that they only consider intervention units excluding u°. Constraint (10) requires that after applying both the contingency set and u°, every query match is destroyed. That is, every query match w must lose at least one edge either because it is deleted by the contingency set or because it is deleted by u°. Constraint (11) selects one query match as the surviving match before u° is deleted. Constraint (12) ensures that the selected query match can be deleted by u°. Finally, constraint (13) ensures that no edge of the selected match is deleted by the contingency set.

Objective. The objective of P-RSP MILP is to minimize the size of the contingency set, i.e., the number of intervention units to delete. An optimum k gives the responsibility $1/(1+k)$ of Definition 3, and infeasibility gives responsibility zero; smaller contingency sets correspond to larger responsibility.

Theorem 4 (Correctness of P-RSP MILP)**.** *For a fixed intervention unit $u^\circ \in U$, the P-RSP MILP computes the minimum size of a contingency set for u°.*

Intuitively, as in the P-RES MILP, d_e marks the edges deleted by the contingency set, and z selects a query match that the contingency set leaves intact and u° destroys (proof in Section IX-B).

Example 9. Recall the example shown in Figure 2, assume u_3 is the given intervention unit, i.e., u°, then $b_{(4,9)}, b_{(4,10)}, b_{(9,10)}$ are 1. Assume we select a contingency set as $\{u_1, u_2\}$, by constraints (8) and (9), $d_{(4,9)}, d_{(4,5)}, d_{(5,9)}, d_{(5,10)}, d_{(9,10)}$ are forced to be 1. Thus, constraint (10) is satisfied by above d_e and b_e. Since w_7 overlaps the selected unit u_2, constraint (13)

with $d_{(5,9)} = 1$ forces $z_{w_7} = 0$; moreover, w_7 does not overlap u°. Setting $z_{w_6} = 1$ satisfies all constraints: w_6 survives the contingency set and is destroyed by $u^\circ = u_3$, so $\{u_1, u_2\}$ is a valid contingency set.

V. LP Relaxation and Approximation

The MILP formulations above provide exact solutions for P-RES and P-RSP. However, solving these MILPs exactly can be very expensive when the number of query matches or intervention-pattern matches is large. To make the computation scalable, we next develop Linear Programming (LP) relaxation-based approximation algorithms. We relax the binary variable x_u for both P-RES and P-RSP to fractional variables in $[0, 1]$. Based on the relaxations, we derive randomized rounding with logarithmic approximation guarantees.

A. Relation to Direct Set Cover

For $w \in W$, let $N(w) = \{u \in U : E_u \cap E_w \neq \emptyset\}$. The direct set-cover LP is

$$\min \sum_{u \in U} x_u \text{ s.t. } \sum_{u \in N(w)} x_u \geq 1 \; (w \in W),\; 0 \leq x_u \leq 1 \;\; (u \in U). \tag{17}$$

With binary x, this is an exact alternative to our MILP, since a query match disappears precisely when a selected unit intersects it; for a fixed survivor r of P-RSP, replacing U by U_r and W by $W^0(u^\circ)$ gives the conditioned problem. Thus the set-cover reduction, LP relaxation, and randomized rounding are standard components. The two relaxations, however, differ. Any direct-LP solution extends to a linked-LP solution by setting $d_e = \min\{1, \sum_{u: e \in E_u} x_u\}$, so $LP^*_{\text{linked}} \leq LP^*_{\text{direct}} \leq$ OPT, and the first inequality can be strict: if one unit contains all q edges of the only query match, the direct LP needs $x_u = 1$, whereas the linked LP allows $x_u = d_e = 1/q$. Consequently, the analysis below cannot import the direct constraint $\sum_{u \in N(w)} x_u \geq 1$; it must establish $\sum_{u \in N(w)} x_u \geq 1/q$ from the linking constraints, which explains the factor q in the rounding parameter. The formulations also differ in size. Let $m = |U|$, $n = |W|$, a be the number of edges in some unit or query match, $I_U = \sum_u |E_u|$, $I_W = \sum_w |E_w|$, the number $I_{UW} = \sum_w |N(w)|$ of unit–match pairs sharing an edge, and $L = m + n + a + I_U + I_W$. The linked LP has $m + a$ variables, $a + I_U + n$ constraints, and $O(L)$ nonzeros, and rounding with repair takes $O(L)$ time (Section IX-C); its size is thus linear in the materialized units and matches. The direct LP has m variables and n rows but I_{UW} nonzeros, which can reach $m \cdot n$. Section VI-B compares both.

B. LP Relaxation for P-RES MILP

LP relaxation. For P-RES, we relax the binary variables in constraint (5) to $0 \leq x_u \leq 1$, $\forall u \in U$. All other constraints and the objective remain unchanged. Let (x^*, d^*) be an optimal LP solution and let $LP^*_{\text{P-RES}} = \sum_{u \in U} x^*_u$ be the LP optimum. Since the relaxation enlarges the feasible region of the MILP, $LP^*_{\text{P-RES}} \leq \text{OPT}_{\text{P-RES}}$.

Rounding Algorithm. Algorithm 1 gives our LP-rounding approximation algorithm (LpRdRes) for P-RES. Its design

Algorithm 1 LP-Rounding for P-RES (LpRdRes)

Input: Intervention units U, query matches W, size of fixed query $q = |Q|$, edge sets $\{E_u\}_{u \in U}$, $\{E_w\}_{w \in W}$.
Output: Intervention set $S \subseteq U$.
1: Build $\mathcal{L} \leftarrow \text{LOOKUP}(W, U)$.
Step 1: LP relaxation
2: Solve the LP relaxation of the P-RES MILP and obtain (x^*, d^*).
Step 2: Randomized rounding
3: Initialize $S \leftarrow \emptyset$ and $\lambda \leftarrow \max\left\{1,\; q \ln\left(\frac{n}{LP^*_{\text{P-RES}}}\right)\right\}$.
4: **for** each $u \in U$ **do**
5: Set $p_u \leftarrow \min\{1, 1 - (1 - x^*_u)^\lambda\}$.
6: Add u to S independently with probability p_u.
7: Set $D \leftarrow \bigcup_{u \in S} E_u$.
8: **for** each $w \in W$ **do**
9: **if** $E_w \cap D = \emptyset$ **then**
10: $u_w \leftarrow \text{SELECTUNIT}(w, \mathcal{L})$.
11: Add u_w to S.
12: Update $D \leftarrow D \cup E_{u_w}$.
13: **return** S.

consists of two main steps. Step 1 solves an LP relaxation to obtain fractional guidance for selecting intervention units. Step 2 applies randomized rounding to obtain an integral solution with feasibility repair. The algorithm first builds a sparse lookup structure $\mathcal{L}$ over the materialized query matches and intervention units (line 1). It avoids storing all pairs (u, w) such that $E_u \cap E_w \neq \emptyset$. Details of LOOKUP are given in Algorithm 3 (Section IX-A). In Step 1, the algorithm solves the LP relaxation of the P-RES MILP and obtains the optimal fractional solution (x^*, d^*) (line 2). The values x^*_u quantify the fractional preference for selecting each intervention unit u and guide the subsequent integral solution construction. In Step 2, the algorithm first initializes the output set S and sets the rounding parameter $\lambda = \max\left\{1, q \ln\left(\frac{n}{LP^*_{\text{P-RES}}}\right)\right\}$ (line 3), where $n = |W|$ is the number of query matches. For each intervention unit $u \in U$, it then computes the sampling probability $p_u = \min\{1, 1 - (1 - x^*_u)^\lambda\}$ (line 5) and independently adds u to S with probability p_u (line 6). This probability transformation amplifies larger LP values while keeping the probability in $[0, 1]$. Thus, units that are more important in the LP solution are more likely to be selected during rounding. The second part of Step 2 repairs the rounded set to ensure feasibility. The algorithm computes the deleted-edge set $D = \bigcup_{u \in S} E_u$ (line 7) to check whether any match remains after rounding (line 8- 9). For every such leftover match, the algorithm invokes $\text{SELECTUNIT}(w, \mathcal{L})$ to find one intervention unit u_w whose edge set intersects E_w (line 10). Details of SELECTUNIT are in Algorithm 4 (Section IX-A). The selected unit u_w is then added to the intervention set S (line 11), and the deleted-edge set is updated accordingly (line 12). Finally, it returns S (line 13).

Theorem 5. *For every feasible P-RES instance, LpRdRes returns a feasible intervention set. Under the fixed-query assumption, the expected size of the returned set is an $O(\log n)$-approximation.*

Intuitively, the LP constraints give every query match a fractional coverage of at least $1/q$ by the units intersecting it;

rounding with λ amplifies this coverage, and the repair step guarantees feasibility (proof in Section IX-B).

Example 10. Recall the example shown in Figure 2, assume the LP output of all intervention units are $x^*_{u_1} = 0.63$, $x^*_{u_2} = 0.21$, $x^*_{u_3} = 0.98$, $x^*_{u_4} = 0.12$. Then the LP optimum $LP^*_{\text{P-RES}} = 1.94$. Meanwhile, $q = 4$ and $n = 7$, which gives us $\lambda = 4\ln(7/1.94) \approx 5.13$. Then the probabilities of selecting each intervention unit are 0.99, 0.70, 1.0, and 0.48 for $p_{u_1}, \ldots, p_{u_4}$, respectively. Assume the rounding step selects u_1 and u_3; then w_4 still survives, since its only edge shared with an intervention unit, $(5, 10)$, belongs to u_2 and u_4. The repair procedure selects one unit that eliminates w_4, e.g., u_2. The final intervention set $\{u_1, u_2, u_3\}$ is feasible; the approximation guarantee follows the theorem above.

C. LP-Rounding Approximation for P-RSP

From the MILP to survivor-conditioned LPs. Recall the P-RSP MILP from Section IV-B, where we select one query match as the *surviving* match, s.t. it can *not* be deleted by the contingency set, but can be deleted by the given intervention unit u°. The selection of the surviving query match is managed by z_w.

An intuition of an approximation algorithm for P-RSP is to relax the P-RSP MILP. However, a direct LP relaxation of the P-RSP MILP would relax $x_u \in \{0, 1\}$ to $0 \le x_u \le 1$ while retaining the fractional variables d_e and z_w. This relaxation is not directly suitable for rounding: the constraint $\sum_{w \in W} z_w = 1$ allows the surviving match's mass to be distributed fractionally among multiple matches, without guaranteeing that any single match is completely preserved.

Therefore, we instead exploit the disjunctive structure encoded by the surviving match variables in the P-RSP MILP. Given the candidate cause u°, define

$$W^+(u^\circ) = \{r \in W \mid E_r \cap E_{u^\circ} \neq \emptyset\}.$$

These are the query matches that can be destroyed by u° and are therefore possible surviving matches before u° is deleted.

In an integral solution of the P-RSP MILP, the binary variables x_u force every d_e to be integral. Consequently, if $z_r > 0$, constraint (13) gives $d_e = 0$, $\forall e \in E_r$. Thus, r genuinely survives the contingency set. Moreover, constraint (12) ensures that $r \in W^+(u^\circ)$. We therefore fix one such match r as the survivor by setting $z_r = 1$ and $z_w = 0$ for every $w \neq r$. It follows that the P-RSP MILP admits the exact decomposition

$$\mathsf{OPT}_{\text{P-RSP}} = \min_{r \in W^+(u^\circ)} \mathsf{OPT}_{\text{P-RSP}}(r),$$

where $\mathsf{OPT}_{\text{P-RSP}}(r)$ is the minimum contingency-set size under the condition that r survives.

Conditioning on a surviving match. Fix a candidate surviving match $r \in W^+(u^\circ)$. Setting $z_r = 1$ in constraint (13) gives $d_e = 0$, $\forall e \in E_r$. Together with constraint (9), this implies $x_u = 0$ for every intervention unit whose edge set overlaps with E_r. We therefore define the intervention units that preserve r as

$$U_r = \{u \in U \setminus \{u^\circ\} \mid E_u \cap E_r = \emptyset\}.$$

We also define

$$W^0(u^\circ) = \{w \in W \mid E_w \cap E_{u^\circ} = \emptyset\}.$$

Every match in $W^0(u^\circ)$ is unaffected by u° and must therefore be destroyed by the contingency set. For every match $w \notin W^0(u^\circ)$, we have $\sum_{e \in E_w} b_e \ge 1$, so constraint (10) is automatically satisfied after u° is deleted.

Hence, after fixing r, the original P-RSP MILP reduces to the following r-conditioned formulation:

$$\begin{aligned}
\min \quad & \sum_{u \in U_r} x_u && && (18)\\
\text{s.t.} \quad & d_e \le \sum_{\substack{u \in U_r \\ e \in E_u}} x_u && \forall e \in E, && (19)\\
& d_e \ge x_u && \forall u \in U_r,\ \forall e \in E_u, && (20)\\
& \sum_{e \in E_w} d_e \ge 1 && \forall w \in W^0(u^\circ), && (21)\\
& x_u \in \{0, 1\} && \forall u \in U_r, && (22)\\
& 0 \le d_e \le 1 && \forall e \in E. && (23)
\end{aligned}$$

This formulation resembles the P-RES MILP, but has two restrictions that are specific to responsibility. First, only the matches in $W^0(u^\circ)$ must be destroyed by the contingency set. Second, only intervention units in U_r may be selected, ensuring that the designated surviving match r is preserved.

LP relaxation. For each $r \in W^+(u^\circ)$, we relax constraint (22) to $0 \le x_u \le 1$, $\forall u \in U_r$. All other constraints and the objective remain unchanged. Let $(x^{r,*}, d^{r,*})$ be an optimal solution and let $LP^*_{\text{P-RSP}}(r) = \sum_{u \in U_r} x^{r,*}_u$ be the optimum of the r-conditioned LP. The overall LP lower bound is $LP^*_{\text{P-RSP}} = \min_{r \in W^+(u^\circ)} LP^*_{\text{P-RSP}}(r)$, where infeasible conditioned LPs are ignored.

Lemma 1. *For every feasible P-RSP instance, $LP^*_{\text{P-RSP}} \le \mathsf{OPT}_{\text{P-RSP}}$.*

Proof sketch. Let S^* be an optimal contingency set. Since S^* makes u° a counterfactual cause, there exists a match r^* that survives after deleting S^* but is destroyed after additionally deleting u°. Therefore, $E_{r^*} \cap E_{u^\circ} \neq \emptyset$, and hence $r^* \in W^+(u^\circ)$. Since r^* survives S^*, no unit in S^* intersects r^*. Thus, $S^* \subseteq U_{r^*}$. Moreover, every match in $W^0(u^\circ)$ is unaffected by u° and must already be destroyed by S^*. Therefore, S^* induces a feasible integral solution to the r^*-conditioned formulation with objective value $|S^*|$. It follows that $LP^*_{\text{P-RSP}} \le LP^*_{\text{P-RSP}}(r^*) \le |S^*| = \mathsf{OPT}_{\text{P-RSP}}$. □

Exact pruning of survivor conditions.

Lemma 2 (Equivalent and dominated conditions)**.** *Let $r_1, r_2 \in W^+(u^\circ)$. If $U_{r_1} = U_{r_2}$, their conditioned optimization problems are identical. If $U_{r_1} \subseteq U_{r_2}$, then $\mathsf{OPT}_{\text{P-RSP}}(r_2) \le$*

$\mathsf{OPT}_{\mathsf{P\text{-}RSP}}(r_1)$. *Retaining one representative of each inclusion-maximal compatible set preserves the minimum contingency cardinality.*

Proof. Every condition covers the same fixed set $W^0(u^\circ)$. Equality of the available unit sets therefore gives identical feasible selections and costs. Under inclusion, every selection feasible using U_{r_1} is also available using U_{r_2}, covers the same matches, and preserves r_2 by the definition of U_{r_2}. Thus its cost does not increase. In the finite family, every discarded set is contained in a retained maximal set, so replacing its condition preserves an optimum. The argument permits ties; it does not claim that discarded conditions are strictly suboptimal. □

Once a feasible contingency of size B is known, a condition with a valid LP lower bound at least B cannot improve the incumbent and can be skipped. A solver's merely feasible primal value is not a lower-bound certificate. This rule can skip rounding after an optimal LP solve, or skip further solving when a certified lower bound is already available.

Rounding Algorithm. Algorithm 2 (LpRdRsp) gives our LP rounding approximation algorithm for P-RSP. The algorithm first builds a sparse lookup structure $\mathcal{L}$ over the query matches and intervention units (line 1). The lookup structure supports edge-overlap checks and repair-unit selection without materializing the complete intervention-unit–match incidence matrix. The algorithm partitions the query matches into W^+, containing the matches destroyed by u°, and W^0, containing the matches unaffected by u° (lines 2–3). If W^+ is empty, no match can certify that u° is necessary, so the instance is infeasible (lines 4– 5). If W^0 is empty, deleting u° alone destroys every query match, and the empty contingency set is optimal (lines 6– 7). Next, the algorithm deduplicates compatible sets, removes inclusion-dominated conditions, and enumerates the retained representatives as the match that survives the contingency set (line 10). For each candidate r, LpRdRsp consists of two main steps: it first constructs U_r, containing only intervention units that preserve r (line 11), and solves the corresponding r-conditioned LP (line 12). An infeasible LP means that r cannot serve as the surviving match and is skipped. Secondly, for every feasible conditioned LP, LpRdRsp independently selects each $u \in U_r$ with probability $p_u = \min\{1, 1-(1-x_u^{r,*})^{\lambda_r}\}$, where $\lambda_r = \max\left\{1,\, q\ln\left(\frac{|W^0|}{LP^*_{\mathsf{P\text{-}RSP}}(r)}\right)\right\}$. Since every rounded unit belongs to U_r, randomized rounding cannot destroy the candidate surviving match r. After rounding, the algorithm computes the deleted-edge set D_r (line 20). For every match in W^0 that remains undestroyed, it selects one repair unit from U_r that intersects the match (lines 21– 24). Such a unit exists whenever the conditioned LP is feasible. Finally, LpRdRsp returns the smallest contingency set obtained over all candidate surviving matches (lines 26– 30).

Theorem 6. *For every feasible P-RSP instance, LpRdRsp returns a valid contingency set. Under the fixed-query assumption, the expected size of the returned set is an $O(\log n)$-approximation.*

Algorithm 2 LP-Rounding for P-RSP (LpRdRsp)

Input: Intervention units U, candidate cause u°, query matches W, fixed query size $q = |Q|$, edge sets $\{E_u\}_{u\in U}$ and $\{E_w\}_{w\in W}$.
Output: Contingency set $S \subseteq U \setminus \{u^\circ\}$, or INFEASIBLE.

1: Build $\mathcal{L} \leftarrow \textsc{Lookup}(W, U)$.
2: $W^+ \leftarrow \{r \in W \mid E_r \cap E_{u^\circ} \neq \emptyset\}$.
3: $W^0 \leftarrow \{w \in W \mid E_w \cap E_{u^\circ} = \emptyset\}$.
4: **if** $W^+ = \emptyset$ **then**
5: **return** INFEASIBLE.
6: **if** $W^0 = \emptyset$ **then**
7: **return** $\emptyset$.
8: $S^* \leftarrow \perp$.
9: Compute U_r for all $r \in W^+$; retain representatives R of the inclusion-maximal distinct sets (Lemma 2).
10: **for** each retained $r \in R$ **do**
11: $U_r \leftarrow \{u \in U \setminus \{u^\circ\} \mid E_u \cap E_r = \emptyset\}$.
Step 1: LP Relaxation
12: Solve the r-conditioned LP and obtain $(x^{r,*}, d^{r,*})$.
13: **if** the LP is infeasible **then**
14: **continue**.
15: If $S^* \neq \perp$ and $LP^*_{\mathsf{P\text{-}RSP}}(r) \geq |S^*|$, skip this condition.
16: $\lambda_r \leftarrow \max\left\{1,\, q\ln\left(\frac{|W^0|}{LP^*_{\mathsf{P\text{-}RSP}}(r)}\right)\right\}$. $S_r \leftarrow \emptyset$.
Step 2: Randomized rounding
17: **for** each $u \in U_r$ **do**
18: $p_u \leftarrow \min\{1, 1-(1-x_u^{r,*})^{\lambda_r}\}$.
19: Add u to S_r independently with probability p_u.
20: $D_r \leftarrow \bigcup_{u\in S_r} E_u$.
21: **for** each $w \in W^0$ **do**
22: **if** $E_w \cap D_r = \emptyset$ **then**
23: $u_w \leftarrow \textsc{SelectUnit}(w, \mathcal{L}, U_r)$.
24: $S_r \leftarrow S_r \cup \{u_w\}$.
25: $D_r \leftarrow D_r \cup E_{u_w}$.
26: **if** $S^* = \perp$ or $|S_r| < |S^*|$ **then**
27: $S^* \leftarrow S_r$.
28: **if** $S^* = \perp$ **then**
29: **return** INFEASIBLE.
30: **return** S^*.

Intuitively, the iteration of an optimal survivor rounds a conditioned LP whose value is at most $\mathsf{OPT}_{\mathsf{P\text{-}RSP}}$, and the analysis of Theorem 5 applies to it (proof in Section IX-B).

Example 11. Recall the example in Figure 2, and let $u^\circ = u_1$. Then $W^+ = \{w_1, w_2, w_3, w_7\}$ contains the candidate surviving matches, while $W^0 = \{w_4, w_5, w_6\}$ must be destroyed by the contingency set. Consider the iteration that fixes $r = w_1$ as the survivor. The units preserving w_1 are $U_r = \{u_2, u_3\}$, and the conditioned LP gives $x^{r,*}_{u_2} = x^{r,*}_{u_3} = 1$, with $LP^*_{\mathsf{P\text{-}RSP}}(r) = 2$. Since $q = 4$ and $|W^0| = 3$, we obtain $\lambda_r \approx 1.62$, and both units are selected with probability 1. The resulting contingency set $S_r = \{u_2, u_3\}$ destroys w_4, w_5, w_6 while preserving w_1. After additionally deleting u_1, all remaining matches, including w_1, are destroyed. The algorithm similarly examines w_2, w_3, and w_7 as survivors and returns the smallest feasible set, which has size 2.

VI. Experimental Study

We evaluate four questions: **(Q1)** solution quality for P-RES and P-RSP, **(Q2)** peak memory and time when units overlap many query matches, **(Q3)** optimization and end-to-end runtime, and **(Q4)** scalability on native workloads

TABLE I
STATISTICS OF DATASETS; AVE_DEG=$2|E|/|V|$.

dataset	# nodes	# edges	ave_deg	# node labels	# edge labels
StarWars	260	611	4.7	6	4
wwc2019	2,486	14,799	11.9	5	9
StackOverflow	6,193	11,540	3.7	5	6
Fincen	7,524	40,835	10.9	3	5
ILPG	411,787	1,117,528	5.4	5	10
ICIJ	2,016,523	3,339,267	3.3	5	14
LDBC	282,386,021	1,775,513,185	12.6	8	22

without a cap on intervention units. The appendix reports time and space costs (Section IX-C), intervention granularities (Section IX-D), the rounding strategy (Section IX-E), a high-survivor stress test (Section IX-F), end-to-end cost (Section IX-G), the native workloads (Section IX-H), details of the case study (Section IX-I); Section VI-F compares pattern-based with edge-based intervention in Neo4j.

Environment. We implement all algorithms in Python 3.11.9. LP and MILP models are constructed using PuLP [37] and solved using the COIN-OR Branch-and-Cut (CBC) solver [38]. Query matches and intervention units are retrieved from Neo4j 5.26.0. All experiments are conducted on a machine equipped with a 24-core AMD EPYC 9224 processor, 512 GB of RAM, and 1 TB of disk storage. **Codes are available in [39]**. Randomized methods use the fixed seeds 11, 23, 47, 71, and 97.

A. Experimental Setup

Datasets. Table I summarizes six real property graphs: StarWars (characters and planets) [40], [41], wwc2019 (players and matches) [25], StackOverflow (users, questions, and answers) [42], Fincen (financial transactions) [43], [44], ILPG (legislation and legal references) [45], [46], and ICIJ (offshore entities and their connections) [47], [48]. Scalability also uses a synthetic LDBC graph with 1.78 billion edges [49], [50].

Algorithms. We compare LpRdRes and LpRdRsp with LP-guided greedy (LpGdRes, LpGdRsp) and random (LpRmRes, LpRmRsp) selection, and with direct greedy (GdRes, GdRsp) and random (RmRes, RmRsp) baselines. The LP-guided methods share the same relaxation and differ in their selection strategy; direct methods use no LP. Greedy uses inverted lists, a lazy priority queue, incremental coverage counters, and reverse deletion. As a reference, we solve the exact MILPs of Section IV with CBC; they provide optima for small instances (Section VI-B).

Evaluation Metrics. We report returned intervention-set sizes for P-RES, contingency-set sizes for P-RSP, and runtime. Smaller sets indicate better feasible solutions; a contingency of size k gives responsibility $1/(1+k)$. Returned sizes need not be optimal unless certified. Randomized results are means over five runs. Figures report optimization runtime on materialized inputs; Section IX-G includes materialization.

Query and Intervention Patterns. Both query and intervention patterns specify topological and label constraints together with predicates over vertex and edge properties; a subgraph match is retained only when all these constraints are satisfied. Different pattern selections produce different numbers of query matches and intervention units. We therefore evaluate the algorithms across configurations of increasing size. The query patterns are selected to produce match counts at powers of ten, while the total number of intervention units is bounded by one tenth of the corresponding number of query matches. The complete pattern configurations are in our code repository [39].

B. Q1. Effectiveness

We compare returned set sizes with direct and LP-guided baselines, then assess optimality on small native instances.

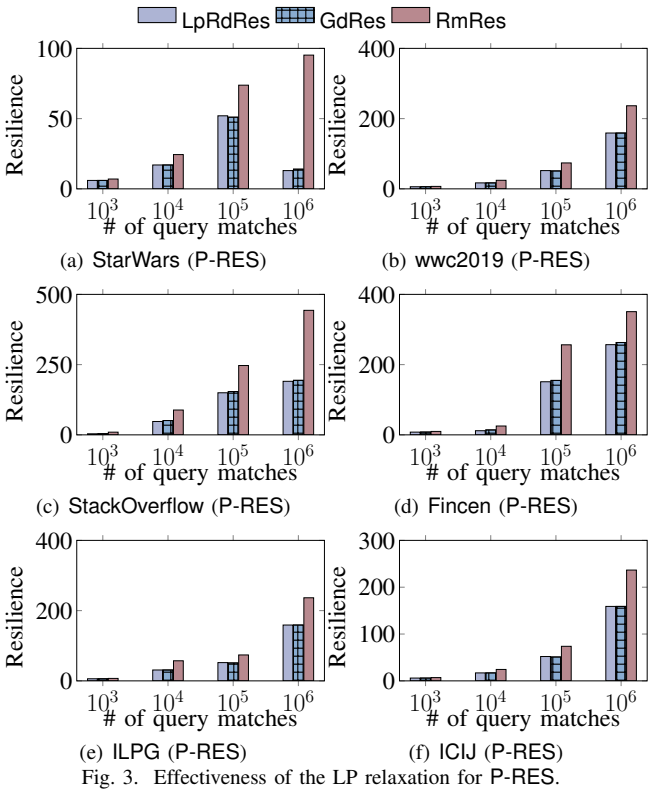


Fig. 3. Effectiveness of the LP relaxation for P-RES.

Effect of LP guidance. Figures 3 and 4 compare LP rounding with direct greedy and random selection. For P-RES, LP rounding and greedy tie at the first two settings on StarWars, wwc2019, ILPG, and ICIJ; rounding uses 52 rather than 51 units at the third setting. At the largest setting they tie on wwc2019, ILPG, and ICIJ, while rounding uses 13 rather than 14 units on StarWars. On StackOverflow and Fincen, rounding returns smaller mean sets at every setting. For P-RSP, the methods tie at the first two settings, rounding uses 51 rather than 50 units at the third, and they tie at the largest except on StarWars (14 versus 15). At the largest settings, direct-random sets are $1.37\times$–$7.32\times$ the LP-rounding sizes for P-RES and $1.52\times$–$6.81\times$ for P-RSP. Thus LP rounding improves substantially over direct random selection, while remaining close to optimized greedy. The random-to-rounding ratio need not increase monotonically with workload size. Holding the LP relaxation fixed, the proposed rounding returns sets no larger than LP-greedy selection and $1.07\times$–$2.42\times$ smaller than LP-random selection (Section IX-E).

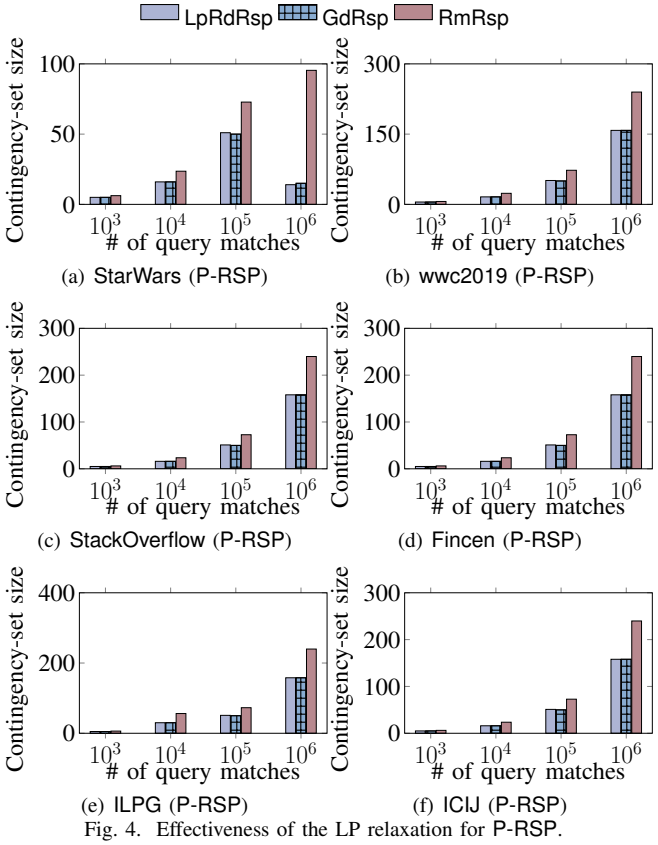


Fig. 4. Effectiveness of the LP relaxation for P-RSP.

Exact Optima and Validity. We solve the exact MILPs on the native instances with up to 10^4 matches. CBC proves optimality in 55 of 70 instances within the 600-second limit per instance. On these 55 instances, optimized greedy attains the optimum in 54, LpRdRes (rounding the linked LP) in 48, and the same rounding applied to the direct set-cover LP of Section V-A in 44; their mean ratios to the optimum (rounded to three decimals) are 1.000, 1.003, and 1.026, with maxima 1.001, 1.125, and 1.325. For single-edge patterns, the MILP agrees with an independent edge-deletion ILP: on all 36 such native instances with an exact solution, both return the same proven optimum. We also verified 856 returned solutions directly in Neo4j on instances with up to 10^3 matches: every resilience solution destroyed all query matches, and every contingency set preserved a match that the designated unit then destroyed (Section IX-D).

Responsibility Diagnostics. For 265 natural targets sampled from native instances and patterns, $|W^+|$ reaches 4,155 (median 17). Deduplication and dominance reduce the 29,411 survivor conditions to 9,585; the LP bound skips 1,325 roundings and the global bound stops 6 searches early. Under a 900-second budget per target and method, 18 targets are censored and reported as such. Among the 164 targets whose optimum CBC certifies, rounding attains it in 151 and greedy in 161. On the target with the largest $|W^+|$ of each of the eight largest overlap-heavy native instances, LpRdRsp returns a contingency $1.3\times$–$7.5\times$ faster than optimized greedy using the same conditions (e.g., 31 versus 230 seconds on Fincen pairs at 10^5 matches, and 1,375 versus 3,911 seconds on Fincen triples at 10^4, without a time budget). The contingency sizes are equal on five targets, smaller on two (e.g., 388 versus 394 on ICIJ triples), and larger on one (4,210 versus 4,203 on ICIJ pairs). Section IX-F stresses up to 96,577 surviving matches.

C. Q2. Memory and Time under Overlap

Optimized greedy keeps the unit–match pairs in memory, whereas LP rounding keeps the edge incidences of the linked LP. To vary the overlap, we restrict the anchors of the native ICIJ, Fincen, and LDBC queries of Table VI (Section VI-E) to a degree range δ (officers of an entity, filings of an originator, replies to a post), add an LDBC query for two members of the same forum with its moderator, and include patterns of up to four participation edges. Each method runs in its own process; we report its peak resident memory above the memory of the process at start. Table II shows representative runs. Greedy's memory grows with the I_{UW} unit–match pairs it stores, whereas LP rounding's grows with the $I_U + I_W$ edge incidences plus the solver's workspace; the memory ratio therefore grows with $r = I_{UW}/(I_U + I_W)$, though not proportionally. For single-edge patterns, greedy needs less memory, since the LP solver's overhead dominates (0.03 versus 0.24 GiB). When units overlap many query matches, greedy needs up to $108\times$ more memory than LP rounding, e.g., 231 versus 5.0 GiB for four officers of entities with 31 to 100 officers, and 105 versus 1.3 GiB for two members of forums with more than 1,000 members. On three officers of the entities with more than 300 officers, greedy has 9.3×10^9 unit–match pairs and was killed by the operating system on the 512 GB machine (at 10^4 matches it already needed 42 GiB), whereas LP rounding finishes in 71 seconds with 3.3 GiB. On these workloads, LP rounding is also $2\times$–$19\times$ faster, with slightly larger solutions.

D. Q3. Efficiency

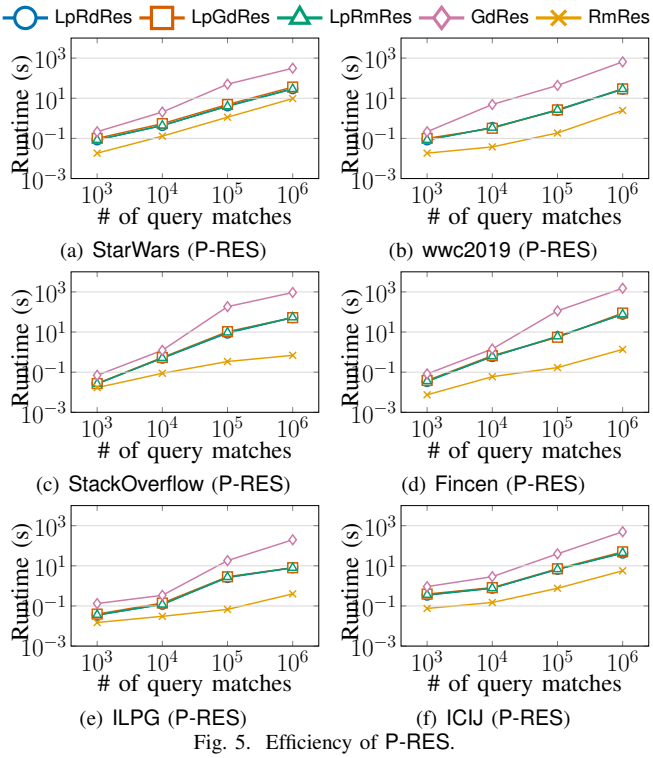
Figures 5 and 6 show that all methods finish quickly on small inputs. At the largest plotted setting of each dataset, LP rounding is $10.5\times$–$24.8\times$ faster than direct greedy for P-RES and $11.8\times$–$22.2\times$ faster for P-RSP. The LP-guided methods have comparable runtimes, with no uniformly fastest rounding strategy. Direct greedy materializes and updates unit–match pairs, whereas LP rounding uses factorized edge incidences.

Direct random selection is fastest throughout the plotted configurations. At the largest settings, it takes 0.40–9.64 seconds for P-RES and 1.60–21.41 seconds for P-RSP. At the same largest workload sizes, its sets are $1.37\times$–$7.32\times$ larger than LP rounding for P-RES and $1.52\times$–$6.81\times$ for P-RSP, showing the solution-quality cost of its runtime advantage.

End-to-End Cost. Including connection to Neo4j and the materialization of query matches and intervention units, all 48 end-to-end runs succeed. On the larger workloads, materialization in Neo4j dominates optimization; e.g., the 10^6-match ICIJ two-officer pipeline completes in 88.3 seconds, of which LP rounding takes 7.8 (Section IX-G).

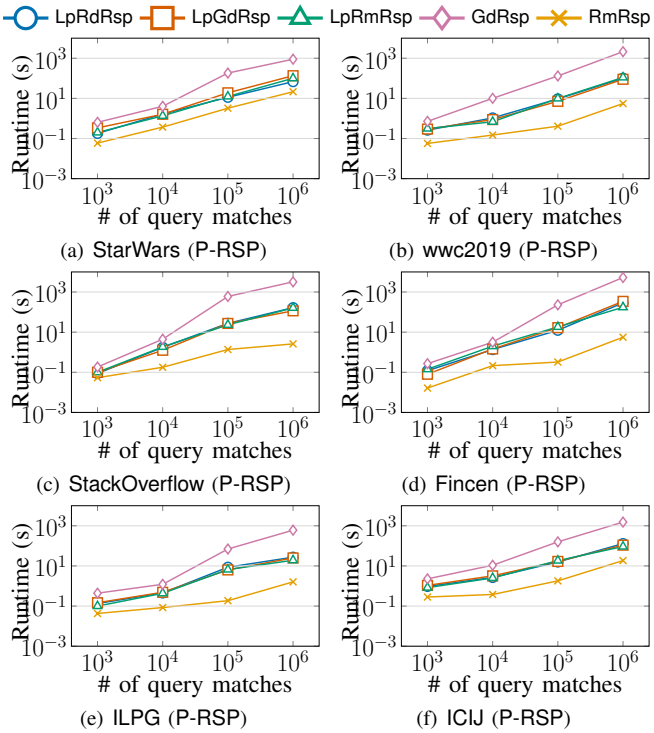
TABLE II
PEAK MEMORY OF GREEDY (GD) AND LP ROUNDING (LPRD); δ: ANCHOR DEGREE; $r = I_{UW}/(I_U+I_W)$; –: GREEDY RAN OUT OF MEMORY.

					Gd			LpRd			
Dataset	Pattern	δ	n	r	$\lvert S\rvert$	time (s)	mem. (GiB)	$\lvert S\rvert$	time (s)	mem. (GiB)	mem. ratio
ICIJ	officer	3–10	10^6	0.6	251,804	7.2	0.03	256,302	1.9	0.24	0.1×
LDBC	4 comments	11–30	10^6	10.0	45,694	84.0	19.7	49,553	42.7	4.0	5.0×
Fincen	3 filings	31–100	10^4	101.6	64	3.5	1.4	75	0.3	0.03	45.2×
ICIJ	4 officers	31–100	10^5	109.2	513	954.3	231.1	652	50.9	5.0	46.3×
LDBC	2 forum members	>1,000	10^6	411.5	831	519.9	105.3	860	60.2	1.3	78.4×
ICIJ	2 officers	>300	10^5	155.1	74	6.2	2.7	83	0.5	0.03	108.2×
ICIJ	3 officers	>300	10^5	–	–	–	–	70	71.3	3.3	–



(a) StarWars (P-RES) (b) wwc2019 (P-RES) (c) StackOverflow (P-RES) (d) Fincen (P-RES) (e) ILPG (P-RES) (f) ICIJ (P-RES)

Fig. 5. Efficiency of P-RES.



(a) StarWars (P-RSP) (b) wwc2019 (P-RSP) (c) StackOverflow (P-RSP) (d) Fincen (P-RSP) (e) ILPG (P-RSP) (f) ICIJ (P-RSP)

Fig. 6. Efficiency of P-RSP.

E. Q4. Scalability

The configurations of Q1 and Q3 share one pair-cover construction and cap the number of units at one tenth of the number of matches. To evaluate scalability on natural structure without this cap, we define one native semantic query per dataset, e.g., two players who played in the same match together with its tournament (wwc2019), and four intervention patterns of increasing density, from a single context edge to groups of three participation edges (Section IX-H). The ratio m/n of intervention units to query matches then ranges from 0.003 to 50, and n reaches 10^6 on ICIJ and on the 1.78-billion-edge LDBC graph. On the largest native instances, LP rounding is 1.14×–5.41× faster than optimized greedy, with equal set sizes in six of nine instances, smaller sizes in two, and a larger size in one (Table VII); the LDBC reply workload takes 1.29 seconds for optimization on 10^6 matches.

F. Case Study: Pattern-based versus Edge-based Intervention

We apply minimum intervention sets in Neo4j under the single-edge pattern P_E of prior work [12] and under intervention patterns chosen by the user, for the query Q of Figure 1(b) on wwc2019 with 10^5 query matches. Besides P_E and the tournament pattern P of Figure 1(b), a single-vertex pattern (Definition 1) denoted P_N below, we use P_2 and P_3, whose units are two or three matches of one tournament with their IN_TOURNAMENT edges (Section IX-I).

Setup. We compute a minimum intervention set S with the exact MILP (proven optimal except under P_3, which reached the solver time limit) and apply it with one Cypher statement per intervention unit, i.e., $\lvert S\rvert$ statements that delete $D(S) = \bigcup_{u\in S} E_u$. We also run greedy selection inside Neo4j, which repeatedly deletes the intervention unit that destroys the most remaining query matches. Every run starts from a freshly restored database and ends with no query match; we report medians over five runs.

TABLE III
MINIMUM INTERVENTION SETS FOR Q (FIGURE 1(B)) ON WWC2019 (10^5 QUERY MATCHES) IN NEO4J; SPEED-UP OVER P_E IN PARENTHESES.

Pattern	$\|S\|$	$\|D(S)\|$	Apply S (s)	Greedy (s)
P_E	284	284	4.66	8.25
P_2	142	284	2.96 (1.6×)	6.89 (1.2×)
P_3	98	284	2.31 (2.0×)	11.68 (0.7×)
P_N	8	284	0.51 (9.1×)	3.60 (2.3×)

Results. All four minimum intervention sets delete exactly the same 284 IN_TOURNAMENT edges, one for each match of the eight tournaments; they differ only in how these edges are grouped into intervention units. Applying S is faster with fewer intervention units: $1.6\times$ under P_2, $2.0\times$ under P_3, and $9.1\times$ under P_N (0.51 versus 4.66 seconds), since a Cypher statement costs 11 to 25 ms in Neo4j but a deleted edge only 0.05 to 0.82 ms. The gain thus comes from grouping the same edges, not from deleting fewer or different ones, and it also holds for multi-edge patterns. Under greedy selection, P_N is $2.3\times$ faster than P_E. More generally, a returned multi-edge unit deletes 1.8 to 3.0 edges on average, e.g., 2,523 co-playing triples delete 7,379 appearance edges on wwc2019 at 10^5 matches (Section IX-D).

VII. RELATED WORK

Necessary Explanations. The notion of necessity and explanation was introduced by Halpern et al. [31], [51], [52] in the context of causal reasoning, where an explanation is defined in terms of counterfactual causality and minimal interventions on the input. This line of work, rooted in causal AI rather than data management [53], formalizes how causes and *responsibilities* can be attributed to outcomes through structural causal models. While our work is inspired by this notion of intervention, we operate in the context of graph databases, where our bulk interventions take the form of subgraph deletions on the underlying graph instance rather than modifications to a causal model. Finding explanations for and understanding the causes of query results is a problem that has also driven data management research [54]–[56]. Many approaches to explanation for relational databases have been explored. Perhaps the most interesting for our problem is modifying the input [57]–[60]. Still, unlike these approaches, which operate on individual relational tuples, our bulk interventions consist of pattern-induced subgraphs.

Resilience and Causal Responsibility. The concept of causal responsibility was further adapted into relational databases [26]–[28], [33] to formalize the notion of query *resilience*. For a Boolean query, *resilience* measures how robust the query outcome is under deletions of input tuples. While resilience quantifies the minimum amount of change needed to flip the query outcome, *responsibility* in relational databases ranks how strongly an individual tuple contributes to the outcome [26], [28], [33]. Our work adapts the concepts of *resilience* and *responsibility* to graph databases. Due to the structural characteristics of graph instances, we extend the notion of intervention unit to a pattern-induced subgraph, which in turn helps discover important structures inside the property graphs. A dichotomy for the complexity of both resilience and responsibility on self-join free queries under set semantics was established in [27]. New results followed on queries with self-joins [61], characterizing the complexity landscape of resilience for conjunctive queries with self-joins and highlighting the structural reasons behind the hardness introduced by self-joins. This is particularly relevant in the context of graph queries, where self-joins arise naturally even in simple graph patterns. Recently, the complexity of resilience for RPQ (regular path queries) in graph databases has been investigated and found tractable for local languages [12], [34]. In our paper, we focus on efficient algorithms to compute both resilience and responsibility for non-recursive graph queries. On materialized matches, resilience is a set-cover problem and conditioning on a surviving match gives the corresponding responsibility subproblem; our LP relaxation and randomized rounding build on these standard tools [26]; our distinct ingredients are the linked unit–edge relaxation with overlap-dependent analysis and compatible-unit restrictions for P-RSP.

ILP for Resilience and Responsibility. A unified approach to compute the resilience and the causal responsibility in relational databases was established in [26] using ILP formulations of the problems, as well as an (MI)LP relaxation. While we also formulate our problems in MILP (which we then relax for approximation purposes), our formulations take into consideration the challenges brought by pattern-based bulk interventions. Since an intervention unit only destroys a query match if they share at least one edge, we introduce new variables and constraints taking into account the concept of subgraph intervention units, and linking these intervention units to the edges that constitute them. Further, our MILP formulations for the optimal P-RES and P-RSP can be reduced to the respective ILPs in the existing work [26] in the special case where the intervention unit is a single edge, since in such a setting, the edges forming the intervention unit are equal to the intervention unit itself.

VIII. CONCLUSION

We introduced pattern-based resilience (P-RES) and pattern-based responsibility (P-RSP), whose intervention units are matches of a user-defined graph pattern. Our exact MILP formulations link intervention units to query matches only through shared edges, so their size grows linearly with the materialized units and matches, and our LP rounding algorithms give an expected $O(\log n)$ approximation for both problems. Experiments show solution sizes close to exact optima and to optimized greedy; where units overlap many query matches, LP rounding needs up to $108\times$ less peak memory than greedy and is up to $19\times$ faster, at the cost of somewhat larger solutions. In future work, we plan to extend pattern-based interventions to recursive queries such as regular path queries, and to avoid materializing query matches by integrating our formulations with query evaluation.

References

[1] S. Sahu, A. Mhedhbi, S. Salihoglu, J. Lin, and M. T. Özsu, "The ubiquity of large graphs and surprising challenges of graph processing: Extended survey," *The VLDB Journal*, vol. 29, pp. 595–618, 2020.

[2] A.-L. Barabási and Z. N. Oltvai, "Network biology: Understanding the cell's functional organization," *Nature Reviews Genetics*, vol. 5, no. 2, pp. 101–113, 2004.

[3] ISO/IEC, "ISO/IEC 39075:2024 Information Technology—Database Languages—GQL," International Organization for Standardization, Tech. Rep. ISO/IEC 39075:2024, 2024. [Online]. Available: https://www.iso.org/standard/76120.html

[4] Neo4j, Inc., "Neo4j Graph Database," accessed: 2026-07-14. [Online]. Available: https://neo4j.com/product/neo4j-graph-database/

[5] Amazon Web Services, "Amazon Neptune," accessed: 2026-07-14. [Online]. Available: https://aws.amazon.com/neptune/

[6] Oracle Corporation, "Oracle Integrated Graph Database," accessed: 2026-07-14. [Online]. Available: https://www.oracle.com/database/integrated-graph-database/

[7] M. Rudolf, M. Paradies, C. Bornhövd, and W. Lehner, "The graph story of the SAP HANA database," in *Proceedings of the 15th Conference on Database Systems for Business, Technology, and Web (BTW)*, ser. Lecture Notes in Informatics, vol. P-214, 2013, pp. 403–420.

[8] RedisGraph Contributors, "RedisGraph: A graph database module for Redis," project no longer maintained; accessed: 2026-07-14. [Online]. Available: https://github.com/RedisGraph/RedisGraph

[9] N. Martínez-Bazán, V. Muntés-Mulero, S. Gómez-Villamor, J. Nin, M. Sánchez-Martínez, and J.-L. Larriba-Pey, "DEX: High-performance exploration on large graphs for information retrieval," in *Proceedings of the 16th ACM Conference on Information and Knowledge Management*, 2007, pp. 573–582.

[10] KuzuDB Contributors, "Kuzu: An embedded property graph database," project archived on October 10, 2025; accessed: 2026-07-14. [Online]. Available: https://github.com/kuzudb/kuzu

[11] ISO/IEC, "ISO/IEC 9075-16:2023 Information Technology—Database Languages SQL—Part 16: Property Graph Queries (SQL/PGQ)," International Organization for Standardization, Tech. Rep. ISO/IEC 9075-16:2023, 2023. [Online]. Available: https://www.iso.org/standard/79473.html

[12] A. Amarilli, W. Gatterbauer, N. Makhija, and M. Monet, "Resilience for regular path queries: Towards a complexity classification," *Proceedings of the ACM on Management of Data*, vol. 3, no. 2, pp. 1–18, 2025.

[13] G. W. Bemis and M. A. Murcko, "The properties of known drugs. 1. molecular frameworks," *Journal of Medicinal Chemistry*, vol. 39, no. 15, pp. 2887–2893, 1996.

[14] T. Chen, D. Qiu, Y. Wu, A. Khan, X. Ke, and Y. Gao, "View-based explanations for graph neural networks," *Proc. ACM Manag. Data*, vol. 2, no. 1, 2024.

[15] E. A. Akartuna, S. D. Johnson, and A. Thornton, "A holistic network analysis of the money laundering threat landscape: Assessing criminal typologies, resilience and implications for disruption," *Journal of Quantitative Criminology*, vol. 41, pp. 173–214, 2025.

[16] S. Qi, B. Tong, J. Hu, H. Lin, Y. Pang, W. Yuan, S. Lyu, Z. Guo, K. Huang, X. Ba, Q. Yin, Y. Shen, Y. Zhou, T. Lv, J. Li, L. Zou, Y. Wu, G. Szárnyas, X. Zhu, W. Chen, and C. Hong, "The LDBC financial benchmark: Transaction workload," *Proc. VLDB Endow.*, vol. 18, no. 9, pp. 3007–3020, 2025. [Online]. Available: https://www.vldb.org/pvldb/vol18/p3007-qi.pdf

[17] J. Blanuša, M. C. Baraja, A. Anghel, L. von Niederhäusern, E. Altman, H. Pozidis, and K. Atasu, "Graph feature preprocessor: Real-time subgraph-based feature extraction for financial crime detection," in *Proceedings of the 5th ACM International Conference on AI in Finance*, 2024.

[18] Z. Wu, J. Wang, H. Du, D. Jiang, Y. Kang, D. Li, P. Pan, Y. Deng, D. Cao, C.-Y. Hsieh, and T. Hou, "Chemistry-intuitive explanation of graph neural networks for molecular property prediction with substructure masking," *Nature Communications*, vol. 14, no. 2585, 2023.

[19] S. M. Wagner and N. Neshat, "Assessing the vulnerability of supply chains using graph theory," *International Journal of Production Economics*, vol. 126, no. 1, pp. 121–129, 2010.

[20] J. Blackhurst, K. S. Dunn, and C. W. Craighead, "Supply chain vulnerability assessment: A network based visualization and clustering analysis approach," *Journal of Purchasing and Supply Management*, vol. 24, no. 1, pp. 21–30, 2018.

[21] S. Noel and S. Jajodia, "Optimal ids sensor placement and alert prioritization using attack graphs," *Journal of Network and Systems Management*, vol. 16, no. 3, pp. 259–275, 2008.

[22] A. Olswang, R. Puzis, and Y. Elovici, "Prioritizing vulnerability patches in large networks," *Expert Systems with Applications*, vol. 188, p. 115992, 2022.

[23] X. Wang, D. Wang, C. Xu, X. He, Y. Cao, and T.-S. Chua, "Explainable reasoning over knowledge graphs for recommendation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019.

[24] Y. Xian, Z. Fu, S. Muthukrishnan, G. de Melo, and Y. Zhang, "Reinforcement knowledge graph reasoning for explainable recommendation," in *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2019.

[25] Neo4j Graph Examples, "Women's world cup 2019 graph example," https://github.com/neo4j-graph-examples/wwc2019, 2019.

[26] N. Makhija and W. Gatterbauer, "A unified approach for resilience and causal responsibility with integer linear programming (ilp) and lp relaxations," *Proceedings of the ACM on Management of Data*, vol. 1, no. 4, 2023.

[27] C. Freire, W. Gatterbauer, N. Immerman, and A. Meliou, "The complexity of resilience and responsibility for self-join-free conjunctive queries," *Proceedings of the VLDB Endowment*, vol. 9, no. 3, pp. 156–167, 2015.

[28] A. Meliou, W. Gatterbauer, J. Y. Halpern, C. Koch, K. F. Moore, and D. Suciu, "Causality in databases," *IEEE Data Engineering Bulletin*, vol. 33, no. 3, pp. 59–67, 2010.

[29] A. Bonifati, G. H. L. Fletcher, H. Voigt, and N. Yakovets, *Querying Graphs*, ser. Synthesis Lectures on Data Management. USA: Morgan & Claypool Publishers, 2018, vol. 10.

[30] A. Pachera, A. Bonifati, and A. Mauri, "User-centric property graph repairs," *Proceedings of the ACM on Management of Data*, vol. 3, no. 1, pp. 1–27, 2025.

[31] J. Y. Halpern and J. Pearl, "Causes and explanations: A structural-model approach. part i: Causes," *The British Journal for the Philosophy of Science*, vol. 56, no. 4, pp. 843–887, 2005.

[32] J. Pearl, *Causality: Models, Reasoning, and Inference*. USA: Cambridge University Press, 2009.

[33] A. Meliou, W. Gatterbauer, K. F. Moore, and D. Suciu, "The complexity of causality and responsibility for query answers and non-answers," *Proceedings of the VLDB Endowment*, vol. 4, no. 1, pp. 34–45, 2010.

[34] M. Bodirsky, v. Semanišinová, and C. Lutz, "The complexity of resilience problems via valued constraint satisfaction problems," in *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science*, ser. LICS '24. New York, NY, USA: Association for Computing Machinery, 2024.

[35] M. R. Garey, D. S. Johnson, and L. Stockmeyer, "Some simplified np-complete graph problems," *Theoretical Computer Science*, vol. 1, no. 3, pp. 237–267, 1976.

[36] C. H. Papadimitriou and M. Yannakakis, "Optimization, approximation, and complexity classes," *Journal of Computer and System Sciences*, vol. 43, no. 3, pp. 425–440, 1991.

[37] S. Mitchell, M. OSullivan, and I. Dunning, "Pulp: a linear programming toolkit for python," *The University of Auckland, Auckland, New Zealand*, vol. 65, no. 25, p. 2011, 2011.

[38] J. J. H. Forrest and R. D. Lougee-Heimer, "Cbc user guide," in *Emerging Theory, Methods, and Applications*. INFORMS, 2005, pp. 257–277.

[39] "Code and datasets," 2026, *https://anonymous.4open.science/r/C4GQ-0507/*.

[40] Neo4j Graph Examples, "Star wars graph example," https://github.com/neo4j-graph-examples/star-wars, 2022.

[41] Kaggle, "The star wars dataverse," https://www.kaggle.com/datasets/jsphyg/star-wars.

[42] Neo4j Graph Examples, "Stack overflow graph example," https://github.com/neo4j-graph-examples/stackoverflow, 2024.

[43] ——, "Icij fincen files graph example," https://github.com/neo4j-graph-examples/fincen, 2020.

[44] E. Díaz-Struck and A. Armendariz, "Download fincen files transaction data," https://www.icij.org/investigations/fincen-files/download-fincen-files-transaction-data/, Sep. 2020.

[45] A. Colombo, A. Bernasconi, and S. Ceri, "Italian legislative property graph," Zenodo dataset, 2024. [Online]. Available: https://doi.org/10.5281/zenodo.11210265

[46] ——, "Modelling legislative systems into property graphs to enable advanced pattern detection," 2024, arXiv:2406.14935. [Online]. Available: https://doi.org/10.48550/arXiv.2406.14935

[47] Neo4j Graph Examples, "Icij offshore leaks graph example," https://github.com/neo4j-graph-examples/icij-offshoreleaks, 2025, accessed: 2026-07-13.
[48] International Consortium of Investigative Journalists, "Icij offshore leaks database," https://offshoreleaks.icij.org/pages/database, 2026, accessed: 2026-07-13.
[49] Linked Data Benchmark Council, "Ldbc social network benchmark data generator," https://github.com/ldbc/ldbc_snb_datagen_hadoop, 2025.
[50] R. Angles, J. B. Antal, A. Averbuch, P. Boncz, O. Erling, A. Gubichev, V. Haprian, M. Kaufmann, J. L. Larriba-Pey, N. Martínez-Bazan, J. Marton, M. Paradies, M.-D. Pham, A. Prat-Pérez, M. Spasic, B. A. Steer, G. Szárnyas, and J. Waudby, "The ldbc social network benchmark," 2020, arXiv:2001.02299. [Online]. Available: https://doi.org/10.48550/arXiv.2001.02299
[51] J. Y. Halpern and J. Pearl, "Causes and explanations: A structural-model approach. part ii: Explanations," *British Journal for the Philosophy of Science*, vol. 56, pp. 889–911, 2005.
[52] H. Chockler and J. Y. Halpern, "Responsibility and blame: A structural-model approach," *Journal of Artificial Intelligence Research*, vol. 22, pp. 93–115, 2004. [Online]. Available: https://doi.org/10.1613/jair.1391
[53] J. Y. Halpern, "Cause, responsibility and blame: a structural-model approach," *Law, Probability and Risk*, vol. 14, no. 2, pp. 91–118, 06 2015. [Online]. Available: https://doi.org/10.1093/lpr/mgu020
[54] B. Glavic, A. Meliou, and S. Roy, "Trends in explanations: Understanding and debugging data-driven systems," *Found. Trends Databases*, vol. 11, no. 3, p. 226318, Aug. 2021. [Online]. Available: https://doi.org/10.1561/1900000074
[55] L. Bertossi, "Specifying and computing causes for query answers in databases via database repairs and repair-programs," *Knowl. Inf. Syst.*, vol. 63, no. 1, p. 199231, Jan. 2021. [Online]. Available: https://doi.org/10.1007/s10115-020-01516-6
[56] L. Bertossi and B. Salimi, "From causes for database queries to repairs and model-based diagnosis and back," *Theor. Comp. Sys.*, vol. 61, no. 1, p. 191232, Jul. 2017. [Online]. Available: https://doi.org/10.1007/s00224-016-9718-9
[57] M. Herschel, M. A. Hernández, and W.-C. Tan, "Artemis: a system for analyzing missing answers," *Proc. VLDB Endow.*, vol. 2, no. 2, p. 15501553, Aug. 2009. [Online]. Available: https://doi.org/10.14778/1687553.1687588
[58] E. Wu and S. Madden, "Scorpion: explaining away outliers in aggregate queries," *Proc. VLDB Endow.*, vol. 6, no. 8, p. 553564, Jun. 2013. [Online]. Available: https://doi.org/10.14778/2536354.2536356
[59] J. Huang, T. Chen, A. Doan, and J. F. Naughton, "On the provenance of non-answers to queries over extracted data," *Proc. VLDB Endow.*, vol. 1, no. 1, p. 736747, Aug. 2008. [Online]. Available: https://doi.org/10.14778/1453856.1453936
[60] S. Roy and D. Suciu, "A formal approach to finding explanations for database queries," in *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 15791590. [Online]. Available: https://doi.org/10.1145/2588555.2588578
[61] C. Freire, W. Gatterbauer, N. Immerman, and A. Meliou, "New results for the complexity of resilience for binary conjunctive queries with self-joins," in *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS'20)*. New York, NY, USA: Association for Computing Machinery, 2020, pp. 271–284. [Online]. Available: https://doi.org/10.1145/3375395.3387647
[62] A. K. Debnath, R. L. L. de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch, "Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. Correlation with molecular orbital energies and hydrophobicity," *Journal of Medicinal Chemistry*, vol. 34, no. 2, pp. 786–797, 1991.
[63] C. Morris, N. M. Kriege, F. Bause, K. Kersting, P. Mutzel, and M. Neumann, "TUDataset: A collection of benchmark datasets for learning with graphs," in *ICML Workshop on Graph Representation Learning and Beyond*, 2020.

IX. Appendix

A. Algorithm Details

Explanation for Fractional z_w. Since x_u remains binary in the MILP, the linking constraints force each d_e to be binary as well. Specifically, if an edge e is contained in a selected intervention unit, then $d_e \geq x_u = 1$, and hence $d_e = 1$. Otherwise, $\sum_{u:e\in E_u} x_u = 0$, so $d_e = 0$.

It remains to consider z_w. Since $\sum_{w\in W} z_w = 1$ and $z_w \geq 0$, there exists at least one match $r \in W$ with $z_r > 0$. For every edge $e \in E_r$, the constraint $z_r + d_e \leq 1$ implies $d_e < 1$. Because d_e is already forced to be binary, we obtain $d_e = 0$. Hence no edge of r is deleted, and r is a surviving query match.

Therefore, any match with positive z_w is a valid survivor. We may choose one such match r, set $z_r = 1$, and set all other $z_w = 0$ without changing x, d, or the objective value. Thus, relaxing z_w to $0 \leq z_w \leq 1$ does not affect the exactness of the MILP, provided that x_u remains binary.

Procedures for Algorithms. The two procedures LOOKUP and SELECTUNIT used for both LpRdRes and LpRdRsp are shown below.

Algorithm 3 LOOKUP(W, U)

Require: Query matches W and intervention units U
Ensure: Sparse edge-to-unit lookup $\mathcal{L}$

```
1:  E_W ← ∅
2:  for all w ∈ W do
3:      E_W ← E_W ∪ E_w
4:  for all e ∈ E_W do
5:      L(e) ← ∅
6:  for all u ∈ U do
7:      for all e ∈ E_u do
8:          if e ∈ E_W then
9:              L(e) ← L(e) ∪ {u}
10: return L
```

The procedure LOOKUP constructs a sparse edge-to-unit index for efficiently identifying intervention units that overlap a query match. It first collects the set $E_W = \bigcup_{w\in W} E_w$ of edges occurring in at least one query match (lines 1–3). For each edge $e \in E_W$, it initializes an empty lookup entry $\mathcal{L}(e)$ (lines 4–5). It then scans every intervention unit $u \in U$ and inserts u into $\mathcal{L}(e)$ for each edge $e \in E_u \cap E_W$ (lines 6–9). Consequently, the resulting lookup satisfies $\mathcal{L}(e) = \{u \in U \mid e \in E_u\}$ for every edge e occurring in a query match. The lookup is returned (line 10). The lookup avoids explicitly materializing the incidence relation between all intervention units and all query matches, which may require $O(|U||W|)$ space. Instead, $\mathcal{L}$ stores only actual edge-to-unit incidences. Assuming constant-time set operations, its construction requires $O\left(\sum_{w\in W} |E_w| + \sum_{u\in U} |E_u|\right)$ time and $O\left(|E_W| + \sum_{e\in E_W} |\mathcal{L}(e)|\right)$ space.

Given an undestroyed query match w, the procedure SELECTUNIT scans its edges (line 1). For an edge $e \in E_w$, every intervention unit $u \in \mathcal{L}(e)$ contains e. Hence, $e \in E_u \cap E_w$, and deleting u destroys w. Therefore, once a nonempty lookup entry $\mathcal{L}(e)$ is encountered, the procedure returns an arbitrary unit from that entry (line 3).

Algorithm 4 SELECTUNIT$(w, \mathcal{L})$

Require: An undestroyed query match w and lookup $\mathcal{L}$.
Ensure: An intervention unit overlapping w.

```
for all e ∈ E_w do
    if L(e) ≠ ∅ then
        return any u ∈ L(e)
```

All units stored in $\mathcal{L}$ are valid candidates because the lookup is constructed from the permitted intervention-unit set U. For responsibility, the same procedure can be used after constructing the lookup from the restricted candidate set.

B. Omitted Proofs

Proof of Theorem 3. Let $S = \{u \in U \mid x_u = 1\}$ be the intervention set selected by any feasible ILP solution. Constraints (2) and (3) enforce $d_e = 1$ if there exists a $u \in S$, s.t. $e \in E_u$. Thus, the variables d_e exactly represent the physical edges deleted by the selected intervention units. Constraint (4) then requires that every query match $w \in W$ contains at least one deleted edge, and hence every query match is destroyed. Therefore, every feasible P-RES MILP solution gives a valid intervention set with the same objective value. Conversely, given any valid intervention set $S \subseteq U$, set $x_u = 1$ iff $u \in S$, and $d_e = 1$ iff $e \in \bigcup_{u \in S} E_u$. These assignments satisfy constraints (2) and (3), and since S destroys every query match, they also satisfy constraint (4). The objective value is exactly $|S|$. Hence the P-RES MILP optimum is equal to the optimal P-RES value. □

Proof of Theorem 4. As in the P-RES MILP, constraints (8) and (9) ensure that $d_e = 1$ exactly for the edges deleted by S. Constraint (10) requires every query match to lose at least one edge after applying both S and u°. Therefore, all query matches are destroyed after deleting the candidate cause. It remains to ensure that u° is necessary. Constraint (11) selects one query match w with $z_w > 0$ as a surviving match before u° is deleted. Constraint (13) ensures that no edge of this selected match is deleted by S, so the query still holds after deleting the contingency set. Constraint (12) ensures that the selected match is overlapped with u° with at least one edge, and hence is destroyed once u° is deleted. Therefore, every feasible P-RSP MILP solution corresponds to a valid contingency set of the same size. Conversely, given any valid contingency set S, set $x_u = 1$ iff $u \in S$. $d_e = 1$ iff e is deleted by S, and choose as $z_w > 0$ a query match that survives S but is destroyed after deleting u°. These assignments satisfy all constraints and have objective value $|S|$. Hence the P-RSP MILP optimum is exactly the minimum contingency-set size. □

Proof of Theorem 5. For a query match $w \in W$, let $N(w) = \{u \in U \mid E_u \cap E_w \neq \emptyset\}$ be the intervention units that can destroy w. From the LP constraints, every query match must receive one unit of fractional edge deletion: $\sum_{e \in E_w} d_e^* \geq 1$. This implies

$$1 \leq \sum_{e \in E_w} d_e^* \leq \sum_{u \in N(w)} |E_u \cap E_w| x_u^* \leq q \sum_{u \in N(w)} x_u^*,$$

since $|E_w| \leq q$. Hence, $\sum_{u \in N(w)} x_u^* \geq \frac{1}{q}$. In the rounding step, each unit u is selected with probability $p_u = 1 - (1 - x_u^*)^\lambda$, where we set $\lambda = \max\left\{1,\ q \ln\left(\frac{n}{LP^*_{\text{P-RES}}}\right)\right\}$. Thus, the probability that w survives the rounding step is

$$\begin{aligned}\Pr[w \text{ survives}] &= \prod_{u \in N(w)} (1 - p_u) = \prod_{u \in N(w)} (1 - x_u^*)^\lambda \\ &\leq \exp\left(-\lambda \sum_{u \in N(w)} x_u^*\right) \leq \frac{LP^*_{\text{P-RES}}}{n}.\end{aligned}$$

Therefore, the expected number of query matches left after rounding is at most $LP^*_{\text{P-RES}}$. The repair step adds at most one intervention unit for each leftover query match, so the expected number of intervention units for repair is at most $LP^*_{\text{P-RES}}$. Meanwhile, since $p_u = 1 - (1 - x_u^*)^\lambda \leq \lambda x_u^*$, the expected number of rounded units is at most $\sum_{u \in U} p_u \leq \lambda \sum_{u \in U} x_u^* = \lambda LP^*_{\text{P-RES}}$. Hence the expected output size is at most $(\lambda+1)LP^*_{\text{P-RES}} \leq (\lambda+1)\mathsf{OPT}_{\text{P-RES}}$. For a nonempty feasible W, constraint (4) implies $LP^*_{\text{P-RES}} \geq 1/q$. Since Q is fixed, $q = O(1)$, and therefore $\lambda = O(\log n)$. Thus the expected approximation ratio is $O(\log n)$. Finally, feasibility follows from the repair step: every query match not destroyed by rounding is explicitly repaired by adding one intervention unit that shares an edge with it. Hence all query matches are eliminated. □

Proof of Theorem 6. Let S^* be an optimal contingency set, and let r^* be a query match that survives S^* but is destroyed after additionally deleting u°. If r^* was removed by equivalence or dominance, replace it by a retained representative whose compatible set contains U_{r^*}; S^* remains feasible for it. If that condition is bound-pruned, the incumbent already has size at most $\mathsf{OPT}_{\text{P-RSP}}$. Consider the iteration of LpRdRsp corresponding to r^*, and let $L = LP^*_{\text{P-RSP}}(r^*)$. By Lemma 1, $L \leq \mathsf{OPT}_{\text{P-RSP}}$. For a match $w \in W^0(u^\circ)$, let $N_{r^*}(w) = \{u \in U_{r^*} \mid E_u \cap E_w \neq \emptyset\}$ be the survivor-compatible intervention units that can destroy w. From the conditioned LP constraints, $\sum_{e \in E_w} d_e^{r^*,*} \geq 1$. This implies

$$1 \leq \sum_{e \in E_w} d_e^{r^*,*} \leq \sum_{u \in N_{r^*}(w)} |E_u \cap E_w| x_u^{r^*,*} \leq q \sum_{u \in N_{r^*}(w)} x_u^{r^*,*}.$$

Hence, $\sum_{u \in N_{r^*}(w)} x_u^{r^*,*} \geq \frac{1}{q}$. In the rounding step, each unit $u \in U_{r^*}$ is selected with probability $p_u = 1 - (1 - x_u^{r^*,*})^{\lambda_{r^*}}$, where $\lambda_{r^*} = \max\left\{1,\ q \ln\left(\frac{|W^0(u^\circ)|}{L}\right)\right\}$. Thus, the probability that w survives the rounding step is

$$\begin{aligned}\Pr[w \text{ survives}] &= \prod_{u \in N_{r^*}(w)} (1 - p_u) = \prod_{u \in N_{r^*}(w)} (1 - x_u^{r^*,*})^{\lambda_{r^*}} \\ &\leq \exp\left(-\lambda_{r^*} \sum_{u \in N_{r^*}(w)} x_u^{r^*,*}\right) \leq \frac{L}{|W^0(u^\circ)|}.\end{aligned}$$

Therefore, the expected number of matches left after rounding is at most L. The repair step adds at most one intervention unit for each leftover match, so the expected number of repair units is also at most L. Meanwhile, since $p_u = 1 - (1 - x_u^{r^*,*})^{\lambda_{r^*}} \le \lambda_{r^*} x_u^{r^*,*}$, the expected number of rounded units is at most $\sum_{u \in U_{r^*}} p_u \le \lambda_{r^*} \sum_{u \in U_{r^*}} x_u^{r^*,*} = \lambda_{r^*} L$. Hence, the expected size of the solution produced in the r^* iteration is at most $(\lambda_{r^*}+1)L \le (\lambda_{r^*}+1)\mathsf{OPT}_{\mathsf{P\text{-}RSP}}$. Since LpRdRsp returns the smallest solution over all candidate survivors, its expected output size is no larger. Moreover, $L \ge 1/q$ whenever $W^0(u^\circ) \neq \emptyset$. Since Q is fixed, $q = O(1)$, and $|W^0(u^\circ)| \le n$, we have $\lambda_{r^*} = O(\log n)$. Thus, the expected approximation ratio is $O(\log n)$. Finally, all selected units belong to U_{r^*}, so r^* survives the contingency set. The repair step destroys every match in $W^0(u^\circ)$, while every remaining match overlaps with u° and is destroyed after u° is additionally deleted. Therefore, the returned set is a valid contingency set for u°. □

C. Time and Space Costs

Let $m = |U|$, $n = |W|$, and $A = \bigcup_u E_u \cup \bigcup_w E_w$ be the active edge set, with $a = |A|$. Define the factorized incidence sizes $I_U = \sum_u |E_u|$ and $I_W = \sum_w |E_w|$, the number $I_{UW} = \sum_w |N(w)|$ of unit–match pairs sharing an edge, and $L = m + n + a + I_U + I_W$. Let T_{mat} denote graph loading and query/unit materialization time. Constructing the sparse edge–unit and edge–match indices takes $O(L)$ time and space after materialization. The linked resilience LP has $m + a$ variables and $a + I_U + n$ structural constraints, with $O(L)$ nonzeros, excluding bounds. If T_{LP} and M_{LP} denote solver time and workspace for that sparse model, its total time is $T_{\mathrm{mat}} + O(L) + T_{\mathrm{LP}}$, and additional optimization space is $O(L) + M_{\mathrm{LP}}$.

For rounding and repair, scan each unit's edge list at most once when adding it, maintain an edge-deletion bitset, and precompute one eligible unit for each edge. Each match can then be checked and repaired by scanning its edges. The post-LP work is $O(L)$; no complete unit–match incidence matrix is needed. This bound assumes the stated indexed implementation.

For responsibility, let $h = |W^+|$, c be the number of distinct compatible sets, and $|R| \le c \le h$ the retained nondominated count. A conservative explicit preprocessing implementation takes $O(hm + I_U I_W + c^2 m)$ time: build the compatibility masks from edge indices, deduplicate them by hashing with equality verification, and compare distinct masks for inclusion. It uses $O(L + hm)$ space. With sequential condition processing, define $L_r = |U_r| + |W^0| + a + \sum_{u \in U_r} |E_u| + \sum_{w \in W^0} |E_w|$. Total time is bounded by

$$T_{\mathrm{mat}} + O(L + hm + I_U I_W + c^2 m) + \sum_{r \in R} \big(O(L_r) + T_{\mathrm{LP},r}\big).$$

Additional optimization space is $O(L+hm) + \max_{r \in R} M_{\mathrm{LP},r}$. Bit-packed masks or streamed construction can improve constants or storage; the above is a conservative bound. In the worst case $|R| = h$, so responsibility can require one separate LP per survivor.

TABLE IV
EXACT OUTPUTS ON MUTAG MOLECULE 47 FOR THE RING-PLUS-NITRO QUERY AND THE CARBON-RING PATTERN.

Ring	In MIN P-RES set	MIN contingency	RSP
R_a (nitro-bearing)	no	$\{R_d\}$	$1/2$
R_b (central)	yes	$\emptyset$	1
R_c (central)	yes	$\emptyset$	1
R_d (nitro-bearing)	no	$\{R_a\}$	$1/2$

D. Intervention Granularity

Same query, complementary outputs. We apply a query that retrieves a carbon ring attached to a nitro group, and the carbon-ring intervention pattern, to the 188 molecules of MUTAG [62], [63]; the query holds on 179. Table IV reports exact outputs for molecule 47, a dinitropyrene with four fused carbon rings. Its resilience is one ring, attained by either central ring, whereas an optimal edge intervention deletes two bonds and has 81 alternatives. The two nitro-bearing rings belong to no minimum resilience solution, yet each has responsibility $1/2$. Across all 179 molecules, rings answer with fewer actions than bonds in 12 (all fused systems) and never with more; these 12 molecules contain 28 rings with responsibility $1/2$ that belong to no minimum resilience solution.

Single-edge specialization. On all 36 native instances with a one-edge pattern and an exact solution, our MILP and an independently implemented edge-deletion ILP return the same proven optimum. We further verified 856 returned solutions directly in Neo4j on instances with up to 10^3 matches: we deleted the selected edges inside a transaction, re-evaluated the query, and rolled back. Every resilience solution destroyed all matches, and every contingency preserved a match that the designated unit then destroyed. **Units versus deleted edges.** For multi-edge patterns, a returned unit deletes 1.8 to 3.0 edges on average; for example, 2,523 co-playing triples delete 7,379 appearance edges on wwc2019 at 10^5 matches. On 200 pairs of laws in ILPG, removing whole provisions severs all article-level citations from one law to another with 7.7 provisions on average, compared with 21.3 citation edges (median ratio 2.6); in 199 pairs, some provision with positive responsibility belongs to no minimum resilience solution. The two objectives therefore answer different questions: $|S|$ counts semantic actions, while $|D(S)|$ counts the physical deletions they imply.

E. Rounding Strategy

Effect of the rounding strategy. Figures 7 and 8 hold the LP relaxation fixed. LP-greedy returns sets equal to or larger than LP rounding: the largest relative differences are 7.7% for P-RES and 3.2% for P-RSP. LP-random sizes are $1.07\times$–$2.42\times$ those of LP rounding for P-RES and $1.08\times$–$1.73\times$ for P-RSP, across the plotted settings. The proposed probabilities $p_u = 1 - (1 - x_u)^{\lambda}$, followed by repair, therefore give smaller sets than LP-random selection in these experiments. These

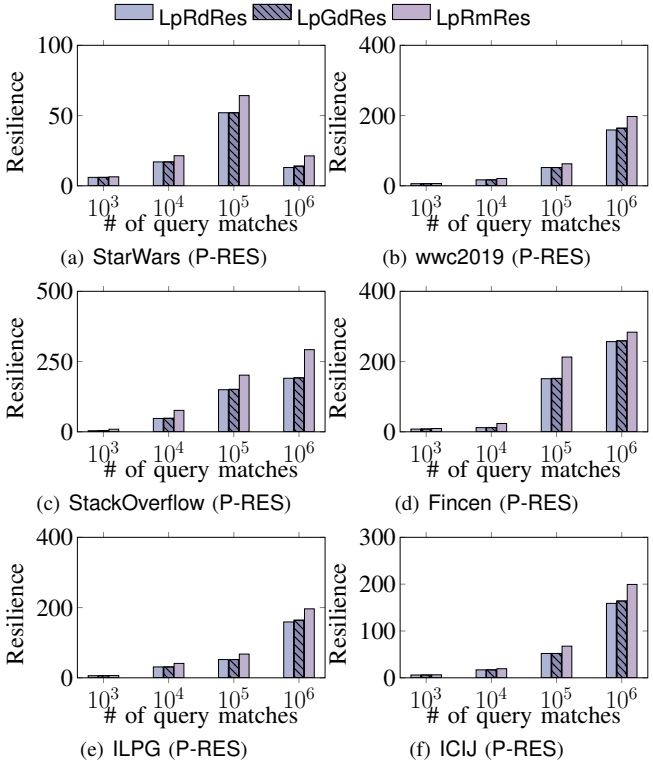


Fig. 7. Effectiveness of the rounding strategy for P-RES.

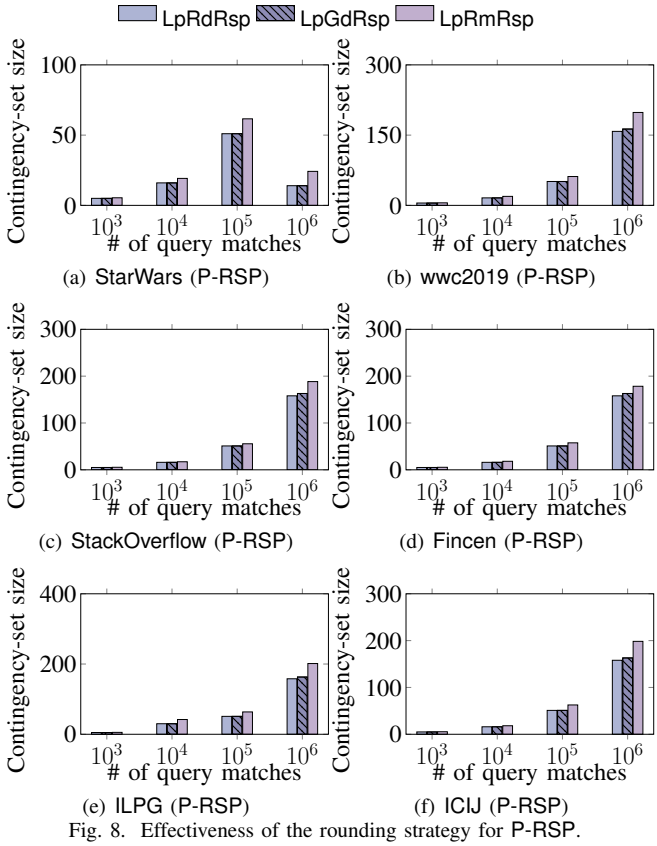


Fig. 8. Effectiveness of the rounding strategy for P-RSP.

aggregate sizes do not separately measure redundancy or repair cost.

F. High-Survivor Stress Test

To test natural targets with many survivors, we take the co-officer query on the 8 ICIJ entities with 200 to 600 officers (845,716 matches). The address unit of the largest entity has $|W^+| = 96{,}577$ surviving matches. Under the address-edge pattern, all 96,577 conditions share one compatible set, so a single conditioned problem is solved. Under the union of officer and address edges, all 96,577 conditions are distinct and mutually incomparable, which is the worst case of Section IX-C; the search nevertheless stops after the first condition, whose local contingency is empty. Under the officer-edge pattern, 1,530 survivors yield 306 distinct conditions. There, LpRdRsp takes 8.8 seconds and optimized greedy 28.0 seconds, with the same contingency size (2,270).

G. End-to-End Cost

We time the complete pipeline in two processes on the same machine. A Neo4j client process checks or starts the database instance and connects (Connect), materializes the matches of Q (Query) and the units of P (Units), builds the edge incidences, and hands them to an optimization process through shared memory. All 48 runs over 16 combinations of dataset, pattern, and n succeed; Table V shows seven of them. Other includes interpreter start-up, imports, incidence construction, and hand-off; rounded phase medians need not sum exactly to the median wall time. Connecting takes 5.3 to 5.5 seconds on a running instance including the instance status check, and a cold start adds 9 to 12 seconds per run. Query materialization grows with n, reaching 36 seconds for 10^6 ICIJ matches. For overlapping pair and triple patterns, unit materialization in Neo4j dominates, e.g., 64.5 seconds for the 840,509 co-playing triples of wwc2019. On the larger listed workloads, query and unit materialization dominate optimization; connection overhead dominates the smallest. The 10^6-match ICIJ two-officer pipeline completes in 88.3 seconds with a running instance.

TABLE V
END-TO-END TIME IN SECONDS (MEDIANS).

Dataset	Pattern	n	Connect	Query	Units	Other	LpRd	Total
StarWars	2 characters	2,365	5.3	0.1	0.1	0.8	0.05	6.4
wwc2019	3 players	10^5	5.4	3.8	64.5	3.1	6.15	82.9
Fincen	2 filings	10^5	5.4	3.5	2.6	1.2	0.63	13.4
ILPG	citing path	10^5	5.3	3.5	7.2	1.7	0.22	18.0
ICIJ	3 officers	10^5	5.3	4.8	49.2	2.6	4.96	66.7
ICIJ	officer	10^6	5.3	35.8	0.0	5.6	1.04	47.7
ICIJ	2 officers	10^6	5.3	35.6	33.5	6.1	7.78	88.3

H. Native Workloads

The configurations above share one pair-cover construction and cap the number of units. To evaluate scalability on natural structure, we define one native semantic query per dataset and four intervention patterns of increasing density (Table VI). Each query is a three-edge pattern, e.g., two players who played in the same match together with the match's tournament (wwc2019), or two officers of the same offshore entity together with its registered address (ICIJ). A coarse

TABLE VI

NATIVE WORKLOADS. PATTERNS ARE LISTED IN COARSE-EDGE / FINE-EDGE / PAIR / TRIPLE ORDER; A COARSE-EDGE UNIT IS A SINGLE CONTEXT EDGE (E.G., TOURNAMENT EDGE: ONE IN_TOURNAMENT EDGE); $n = |W|$, $m = |U|$. THE m/n RANGE COVERS ALL SAMPLED SIZES.

Dataset	Query (three edges)	Intervention patterns	Max n	m/n range
StarWars	Co-appearing characters + homeworld	Homeworld edge / appearance / 2 characters / 3 characters	2,365	0.03–50
wwc2019	Co-playing players + tournament	Tournament edge / appearance / 2 players / 3 players	10^5	0.003–29
StackOverflow	User's answer + question tag	Tag edge / answer / answer–tag / user–answer	3,709	0.33–1.03
Fincen	Filings sharing an originator + filer	Filer edge / originator / 2 filings / 3 filings	10^5	0.02–46
ILPG	Article-to-article citation across laws	Membership edge / citation / citing path / 2 citations	10^5	0.42–44
ICIJ	Co-officers of an entity + address	Address edge / officer / 2 officers / 3 officers	10^6	0.02–7.8
LDBC	Co-replies to a post + post creator	Creator edge / reply / 2 comments / 3 comments	10^6	0.06–2.1

TABLE VII

OPTIMIZATION TIME ON THE LARGEST NATIVE INSTANCES. LB: LOWER BOUND FROM THE REDUCED RELAXATION, ROUNDED UP.

Dataset	Pattern	n	$I_{UW}/(I_U+I_W)$	Gd $\lvert S\rvert$ (s)	LpRd $\lvert S\rvert$ (s)	LB
ICIJ	3 officers	10^5	70.4	3,161 (26.31)	3,138 (4.96)	1,933
ICIJ	2 officers	10^6	16.7	40,519 (13.75)	40,621 (7.78)	27,044
ICIJ	officer	10^6	0.6	73,359 (2.04)	73,359 (1.04)	51,915
Fincen	3 filings	10^4	76.2	69 (12.56)	69 (3.24)	41
Fincen	2 filings	10^5	48.4	1,003 (2.35)	1,003 (0.63)	528
wwc2019	3 players	10^5	22.5	2,523 (11.89)	2,523 (6.15)	1,278
wwc2019	2 players	10^5	10.3	3,783 (1.31)	3,783 (1.15)	1,916
ILPG	citing path	10^5	8.4	22,933 (1.19)	22,916 (0.22)	22,912
LDBC	reply	10^6	0.6	280,495 (4.27)	280,495 (1.29)	168,665

one-edge pattern deletes a shared context edge (e.g., one IN_TOURNAMENT edge on wwc2019; in contrast, a unit of the tournament pattern P_N of Section VI-F contains all IN_TOURNAMENT edges of a tournament), a fine one-edge pattern deletes a single participation edge, and pair and triple patterns delete groups of participation edges, such as two or three players of one match. Matches are sampled by visiting anchors (e.g., matches or entities) in a fixed hashed order up to n matches; units are all pattern instances sharing an edge with a sampled match. Without a cap, m/n ranges from 0.003 to 50.

Table VII reports optimization time from the materialized edge incidences to a validated solution, for selected largest instances. Optimized greedy must materialize the I_{UW} unit–match pairs, whose number exceeds the factorized incidences $I_U + I_W$ by up to $76\times$ for overlapping pair and triple patterns on these instances, and by up to $411\times$ on the degree-restricted workloads of Table II.

On the listed native instances, LP rounding is $1.14\times$–$5.41\times$ faster than greedy, with equal set sizes in six of nine rows, smaller sizes in two, and a larger size in one (40,621 versus 40,519 for ICIJ pairs). The lower bounds do not certify these solutions as optimal. The LDBC reply workload takes 1.29 seconds for optimization on 10^6 materialized matches; this excludes retrieval from the 1.78-billion-edge graph.

I. Case Study Details

On wwc2019, the query Q of Figure 1(b) is the graph pattern $Q_W = (V_\Pi, E_\Pi, \phi_V, \phi_E)$ with

$$
\begin{aligned}
&V_\Pi = \{p, q, m, t\},\\
&E_\Pi = \{r_1{=}(p, m),\ r_2{=}(q, m),\ r_3{=}(m, t)\},\\
&\phi_V:\ \mathsf{Person} \in \lambda(p) \cap \lambda(q),\ \nu(p, \mathsf{id}) < \nu(q, \mathsf{id}),\\
&\qquad \mathsf{Match} \in \lambda(m),\ \mathsf{Tournament} \in \lambda(t),\\
&\phi_E:\ \mathsf{PLAYED_IN} \in \lambda(r_1) \cap \lambda(r_2),\\
&\qquad \mathsf{IN_TOURNAMENT} \in \lambda(r_3),
\end{aligned}
$$

i.e., two players who played in the same match, counted once per pair, together with the tournament of the match. The four intervention patterns of Section VI-F are

$$
\begin{aligned}
P_E &= (\{x, y\}, \{(x, y)\}, \emptyset, \emptyset), \quad E_u = \{e\},\\
P_2 &= (\{m_1, m_2, t\}, \{(m_1, t), (m_2, t)\}, \phi_V, \phi_E),\\
P_3 &= (\{m_1, m_2, m_3, t\},\\
&\qquad \{(m_1, t), (m_2, t), (m_3, t)\}, \phi_V, \phi_E),\\
P_N &= (\{t\}, \emptyset, \mathsf{Tournament} \in \lambda(t), \emptyset),\\
&\qquad E_u = \{e \mid t \in \eta(e),\ \mathsf{IN_TOURNAMENT} \in \lambda(e)\},
\end{aligned}
$$

where ϕ_V requires Match on each m_i and Tournament on t, and ϕ_E requires IN_TOURNAMENT on every edge. Under P_E, every edge is an intervention unit; under P_2 and P_3, an intervention unit consists of two or three matches of one tournament with their IN_TOURNAMENT edges; and P_N, the pattern P of Figure 1(b), is a single-vertex pattern (Definition 1), whose intervention unit is a tournament with all its IN_TOURNAMENT edges.

Greedy selection inside Neo4j re-evaluates Q only on the query matches that lost an edge after each deletion. The per-statement and per-edge costs of Section VI-F come from least-squares fits on the wwc2019 runs of Table III. Under greedy selection, the gain of P_N (3.60 versus 8.25 seconds) is smaller than when applying S, because the initial evaluation of Q, 2.9 seconds, is the same for every pattern.